PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
Departamento de Ingeniería Eléctrica
IEE2913 — Diseño Eléctrico (Capstone)

Grupo 10

Informe de Avance 1 (5 %)

Proyecto: SupaClock: Wearable Biométrico Modular
Integrantes: Tomás Avendaño, Benjamín Sepúlveda, Pablo Uribe
Fecha: 7 de abril de 2026

Índice

1. Contexto y Objetivos **5**

2. Posibles soluciones ante la problemática **5**
2.1. Peto deportivo inteligente . . . 5
2.2. Dispositivo *wearable* tipo *smartband* . . . 6

3. Propuesta de diseño **6**

4. Análisis de requerimientos **7**
4.1. Dispositivo compacto y cómodo . . . 7
4.2. Energización y autonomía . . . 8
4.3. Mediciones requeridas . . . 8
4.4. Procesamiento algorítmico y Machine Learning . . . 8
4.4.1. Contador de pasos algorítmico . . . 8
4.4.2. Clasificación de actividades (4 estados) . . . 9
4.4.3. Justificación de la arquitectura Edge AI . . . 10
4.5. Especificación de manejo de datos . . . 11
4.6. Algoritmos de Procesamiento de Señales (ECG) . . . 11
4.7. Interfaz gráfica y navegación . . . 12

5. Elección de componentes y justificación **13**

6. Diagrama de bloques **23**
6.1. Diagrama de bloques de alto nivel . . . 23
6.2. Diagrama de potencia . . . 25
6.3. Diagrama de señales y buses de comunicación . . . 25
6.3.1. Mapeo de pines y plan de migración (C3 a S3) . . . 26
6.4. Arquitectura de Firmware (FreeRTOS) . . . 27
6.4.1. Frecuencias de Muestreo y Optimización de Memoria . . . 28
6.5. Arquitectura de Ecosistema (App y Servidor) . . . 28
6.5.1. Algoritmos de Procesamiento de Señales (ECG en Servidor) . . . 29

7. Bosquejo de la solución **30**

8. Metodología de trabajo **30**
8.1. Áreas y roles . . . 31
8.1.1. Área 1: Hardware . . . 31
8.1.2. Área 2: Software y Procesamiento . . . 31
8.1.3. Área 3: Interfaz y Sistema . . . 31
8.2. Trabajo colaborativo . . . 32
8.3. Asignación de tareas . . . 32

9. Objetivos a cumplir **33**
9.1. Hitos . . . 33
9.1.1. Hito 5 %: Definición y planificación inicial . . . 33
9.1.2. Hito 25 %: Validación de subsistemas iniciales . . . 33
9.1.3. Hito 60 %: Integración funcional del sistema . . . 34
9.1.4. Hito 90 %: Validación avanzada y optimización . . . 34
9.1.5. Hito 100 %: Entrega final . . . 34

10. Presupuesto estimado **35**

Informe de Avance 1 (5 %) **37**

11. Diagramas de bloques de bajo nivel actualizados **37**
11.1. Arquitectura de Firmware (FreeRTOS) . . . 37
11.2. Pipeline de Datos Biomédicos . . . 38

12. Planificación actualizada **38**
12.1. Estado de avance respecto a la planificación . . . 38
12.2. Actualización de la Carta Gantt . . . 39
12.3. Distribución del trabajo . . . 40

13. Implementación y resultados **40**
13.1. Bloque de Interfaz Local: Driver de pantalla ST7789 . . . 40
13.1.1. Descripción y funcionalidad . . . 40
13.1.2. Integración con LVGL 8.4 . . . 40
13.1.3. Resultados . . . 41
13.2. Bloque de Interfaz Local: Decodificación JPEG (JPEGDEC) . . . 41
13.2.1. Descripción y funcionalidad . . . 41
13.2.2. Resultados . . . 41
13.3. Bloque de Sensores: Prueba del MAX30102 (PPG) . . . 41
13.3.1. Descripción y funcionalidad . . . 41
13.3.2. Resultados . . . 41
13.4. Bloque de Comunicación: BLE con NimBLE . . . 41
13.4.1. Descripción y funcionalidad . . . 41
13.4.2. Resultados . . . 42
13.5. Bloque de Backend: Firebase y Cloud Firestore . . . 42
13.5.1. Descripción y funcionalidad . . . 42
13.5.2. Resultados . . . 42
13.6. Evaluación general y próximos pasos . . . 42

14. Referencias **43**

15. Anexos **45**
15.1. Anexo A: Código del driver ST7789 (extracto principal) 45
15.2. Anexo B: Callback de decodificación JPEG 46
15.3. Anexo C: Driver MAX30102 (lectura FIFO) 46
15.4. Anexo D: Extracto de Cloud Functions (Pan-Tompkins) 46

Índice de figuras

1. Peto deportivo de referencia . . . 5
2. *Smartband* comercial de referencia . . . 6
3. Diseño preliminar de las interfaces gráficas del dispositivo. . . . 12
4. Diagrama de bloques de alto nivel del sistema SupaClock, incluyendo el dispositivo, la aplicación móvil y el backend en la nube. . . . 24
5. Diagrama de flujo de potencia y gestión de batería del sistema SupaClock. . . . 25
6. Diagrama de arquitectura de señales, buses lógicos y direcciones I2C. . . . 26
7. Diagrama de bloques del firmware y gestión de tareas en FreeRTOS. . . . 28
8. Arquitectura de ecosistema: flujo de datos desde el dispositivo *Edge* hacia la aplicación móvil y el backend en la nube. . . . 29
9. Bosquejo preliminar del prototipo SupaClock con vistas cenital, frontal, lateral y posterior. 30
10. Carta Gantt del proyecto SupaClock con la planificación temporal de actividades por hito. 35
11. Arquitectura de tareas concurrentes (FreeRTOS) implementada en el ESP32-C3, destacando el flujo de lectura de sensores y visualización gráfica local. . . . 37
12. Pipeline de adquisición y procesamiento de datos biomédicos, modelando hacia el futuro el recorrido completo de la señal. . . . 38

Índice de cuadros

1. Comparación de algoritmos de conteo de pasos para dispositivos wearable. . . . 9
2. Comparación de arquitecturas de ML para clasificación de actividad en microcontroladores [1]. . . . 10
3. Especificación de volumen, formato y destino de los datos del sistema. . . . 11
4. Comparación de Opciones de Microcontroladores (MCU) . . . 14
5. Comparación de Sensores Inerciales (IMU) para Detección de Actividad . . . 15
6. Comparación de Sensores Ópticos (PPG) para HR y SpO2 . . . 16
7. Comparación de Sensores de Temperatura para Aplicaciones Médicas/Wearables . . . 17
8. Comparación de Circuitos Integrados para Carga y Protección (BMS) . . . 18
9. Desglose del consumo energético estimado por componente (valores de hojas de datos). . . 19
10. Análisis de apilamiento mecánico (*stack-up*) para la arquitectura sandwich de doble PCB. 20
11. Comparación de Monitores de Batería (Fuel Gauges) . . . 21
12. Comparación de Reguladores de Voltaje LDO (3.3V) para Fase 2 . . . 22
13. Comparación de Tecnologías de Pantalla para Interfaz de Usuario . . . 23
14. Mapeo definitivo de señales para placas SuperMini. . . . 27
15. Bill of Materials (BOM) con precios reales de adquisición (IVA incl.) para componentes críticos. . . . 36
16. Contraste entre avance esperado (5 %) y avance logrado. Las filas amarillas indican tareas adelantadas respecto a la planificación original. . . . 39

1. Contexto y Objetivos

Según la Encuesta Nacional de Actividad Física [2], apenas un 49.7 % de los hombres y un 40.3 % de las mujeres mayores de 18 años en el país mantienen un nivel de actividad física adecuado. Esta cifra resulta alarmante al considerar que el 42 % de la población adulta presenta sobrepeso [3], un factor de riesgo determinante para el desarrollo de patologías crónicas a largo plazo. Frente a este escenario, surge la necesidad de fomentar hábitos de vida más saludables. Con el fin de contribuir a esta problemática, el presente proyecto propone el diseño y desarrollo de un dispositivo *wearable* de bajo costo que permita a los usuarios monitorizar sus variables biométricas y registrar su evolución física de manera accesible.

2. Posibles soluciones ante la problemática

Para cumplir con los requerimientos solicitados por el cliente, planteamos dos posibles soluciones. Cada una de ellas con ventajas y desventajas en distintos ámbitos.

2.1. Peto deportivo inteligente

Con el fin de proponer una solución que sea poco visible y sea más fácil de utilizar al momento de realizar deporte, planteamos la posibilidad de diseñar un peto deportivo que integrara los distintos sensores y tuviera una pantalla en el pecho. La Fig.1 ilustra un peto deportivo comercial que se utilizó como referencia de este concepto: una prenda ceñida al torso que podría alojar sensores distribuidos sobre el pecho, aprovechando la cercanía al corazón y la estabilidad del contacto con la piel.

Figura 1: Peto deportivo de referencia. Imagen obtenida de Pro:Direct Sport. Disponible en: `https://images.prodirectsport.com/ProductImages/Main/V3_1_Main_0289806.jpg`

Las principales ventajas de este diseño eran principalmente la facilidad al momento de integrar los sensores, debido a que los circuitos no tendrían que ser tan pequeños. Además, algunas mediciones serían más sencillas de realizar, como las del sensor de ECG.

Sin embargo, existían complicaciones técnicas que nos limitaron a seguir pensando esta opción. La primera de ellas es que quizás sería muy complejo adaptar un circuito a un material más flexible y cómodo para el usuario, considerando que si se utilizan cables la posibilidad de aumentar el ruido entre los sensores y el microcontrolador eran muy altas. También, investigando en algunos de los posibles sensores que podíamos utilizar, algunos fueron diseñados para realizar mediciones en la zona de la muñeca. Por último, analizando desde un punto de vista higiénico, notamos que sería dificil permitir el lavado de la prenda sin generar daños a la circuitería.

2.2. Dispositivo *wearable* tipo *smartband*

Una de los requisitos de diseño implicaba la necesidad de utilizar una pantalla que mostrara los datos sensados, por lo que consideramos que una decisión de diseño fundamental para nuestro producto es que el usuario tuviera fácil acceso y visualización de los datos que el dispositivo mide. Es por ello que analizamos la opción de diseñar un dispositivo que se utilizara en la muñeca y pueda ser fácilmente manipulado por el usuario. La Fig.2 presenta una *smartband* comercial que sirvió como referencia para esta alternativa, mostrando el factor de forma compacto, la pantalla integrada y la correa ajustable que caracterizan a este tipo de dispositivos.

Figura 2: *Smartband* comercial de referencia (Prixton AT410). Disponible en: `https://prixton.com/product/smartband-at410/`

El diseño del producto como *smartband* tiene como ventaja que, al existir una gran variedad de dispositivos de la misma índole en el mercado, existe mucha documentación relacionada al diseño de estos dispositivos, incluyendo sensores adaptados para realizar mediciones en la zona de la muñeca dado que el grosor de la piel en esta zona es menor y permite un mejor monitoreo de las señales corporales como la saturación de oxígeno en sangre. Por otra parte, este dispositivo sería menos invasivo y más adaptable a distintos perfiles de usuarios, considerando que nuestro público objetivo puede variar significativamente su contextura física.

Más allá de las ventajas mencionadas anteriormente, existen distintos desafíos que deben ser tomados en cuenta para diesñar este tipo de dispositivos. La primera consideración es que el producto debe ser pequeño para no generar incomodidades al usuario, esto es una limitación a nivel de diseño y de ingeniería, pues se debe considerar que la batería que tendrá el dispositivo será pequeña y no podemos considerar componentes que consuman mucha corriente para maximizar la duración de la batería, considerando que debemos incluir un modelo de *Machine Learning* dentro o fuera del dispositivo, lo cual es una tarea exigente que puede demandar mucha memoria del microcontrolador si se realiza interna o mucho consumo energético si se envían muchos paquetes de forma inalámbrica a un servidor para su posterior análisis. Siguiendo la línea de la comodidad, se vuelve fundamental la necesidad de miniaturizar el dispositivo, principalmente los componentes utilizados para el funcionamiento óptimo de los sensores, utilizando idealmente componentes SMD, lo cual puede ser difícil considerando que el laboratorio de trabajo no tiene instrumentos ni herramientas óptimos para realizar PCB que utilicen estos componentes.

3. Propuesta de diseño

En base al análisis de las opciones anteriores, se optó por el desarrollo de un *wearable* tipo smartwatch debido a la amplia disponibilidad de componentes adaptados para biometría periférica y la facilidad de

interacción directa con el usuario, así como no tener que lidiar con los largos recorridos de las señales de estar situadas muy lejos entre sí físicamente.

El producto a desarrollar es **SupaClock**, un *wearable* enfocado en el monitoreo biométrico y de actividad física. El entregable principal del proyecto consistirá en un dispositivo compacto, cómodo y completamente funcional. La primera iteración contendrá todos los subsistemas y sensores sobre una PCB básica. Luego de verificar el funcionamiento general, se migrará a una PCB personalizada con los circuitos de los sensores, alimentación, microcontrolador y periféricos, así como una carcasa hecha a medida. El sistema será orquestado inicialmente por el microcontrolador ESP32 C3 [4], dependiendo de su desempeño en etapas iniciales de prueba, se considera migrar hacia el ESP32 S3 [5], que tiene varias mejoras significativas, a costa de un mayor consumo energético. Para gestionar el lado de software, se dispondrá del sistema operativo FreeRTOS [6].

Como objetivo extendido (*bonus*), una vez validada la arquitectura principal, se diseñará una versión "Pro". Esta consistirá en una placa PCB miniaturizada de alta densidad, acercándose a un factor de forma de *smartwatch* comercial. De manera complementaria, el ecosistema se expandirá con una aplicación Android que recibirá los datos vía Bluetooth Low Energy (BLE) y los almacenará de forma persistente en una base de datos en la nube (como Firebase), permitiendo un análisis histórico y avanzado de las métricas del usuario.

Para satisfacer los requerimientos técnicos del proyecto, la propuesta de diseño integra múltiples subsistemas. En primer lugar, la adquisición de señales biológicas se realiza mediante el sensor MAX30102 [7] para fotopletismografía (SpO2/HR) y el MAX30205 [8] para la temperatura corporal, exigiéndose un diseño mecánico que garantice un contacto dérmico directo, constante y cómodo en la muñeca. Complementariamente, se implementa un sistema de electrocardiografía monocanal basado en el circuito AD8232 [9], el cual utiliza un arreglo de tres electrodos de contacto seco (pernos M3) distribuidos en el dispositivo; esto permite medir la Derivación I del corazón mediante una interacción sencilla, requiriendo únicamente que el usuario haga contacto con la mano opuesta sobre un perno situado en la carcasa. En cuanto al monitoreo de la actividad, se integra un acelerómetro y giroscopio (BMI160) [10] para la captura de telemetría inercial en 6 ejes, datos que alimentarán un modelo de Machine Learning capaz de distinguir estados físicos como *caminar, correr, reposo*, o detectar situaciones de *emergencia/caída*. Finalmente, la retroalimentación al usuario se resuelve mediante un despliegue gráfico local, incorporando una pantalla LCD a color [11] para renderizar una interfaz fluida, atractiva y prolija, capaz de mostrar métricas numéricas y formas de onda de manera completamente autónoma y sin depender del teléfono móvil.

4. Análisis de requerimientos

4.1. Dispositivo compacto y cómodo

Al escoger un *wearable*, debemos diseñar un dispositivo que no imposibilite el uso de la muñeca, por lo que no puede superar los 80 mm de largo ni 50 mm de ancho. A su vez, la altura del dispositivo puede generar incomodidad, por lo que debemos realizar un diseño que no supere los 30 mm de altura. Por otra parte, el peso es un factor importante, por lo que nuestro dispositivo no puede pesar más de 150 g.

Dada la naturaleza de algunos sensores, es necesario que algunos de estos deban tener contacto directo con la piel del usuario, por lo que es fundamental que estos se encuentren correctamente protegidos y aislados. Considerando los implementos del laboratorio, realizaremos una carcasa impresa en 3D que albergue todos los componentes en su interior, y que permita una interacción segura entre los sensores y el usuario.

4.2. Energización y autonomía

El dispositivo no puede estar energizado a una fuente de poder para su uso cotidiano, por lo que es fundamental el uso de baterías para alimentar al mismo. Para garantizar un uso óptimo y seguro de la misma, se debe incluir un BMS en el diseño. A su vez, es necesario incluir un circuito *battery fuel gauge*, el integrado MAX17043 se encarga de informar mediante I2C el nivel de batería al usuario.

Para ser un dispositivo a la altura de la competencia, es necesario que la batería tenga una duración mínima de 12 horas, por lo que es necesario optimizar los consumos de energía del microcontrolador y los sensores del dispositivo.

4.3. Mediciones requeridas

Para cumplir íntegramente con las especificaciones del proyecto, el hardware del dispositivo debe ser capaz de adquirir, acondicionar y digitalizar cinco variables biométricas y cinemáticas fundamentales, garantizando la entrega de señales limpias para su posterior procesamiento:

- **Temperatura corporal superficial:** Capturada de manera periódica mediante el sensor de grado clínico MAX30205, el cual requiere un diseño mecánico que asegure contacto térmico directo con la piel del usuario.
- **Frecuencia cardíaca y Saturación de oxígeno (SpO2):** Ambas métricas serán estimadas mediante fotopletismografía (PPG) de reflectancia, utilizando el módulo óptico MAX30102 ubicado en la base del dispositivo.
- **Aceleración:** La captura del movimiento lineal espacial (necesaria tanto para el podómetro como para el modelo de clasificación) se realizará mediante la Unidad de Medición Inercial (IMU) BMI160.
- **Electrocardiograma (ECG):** Se requiere la adquisición de la actividad eléctrica del corazón. Esta función estará destinada de manera exclusiva a **mediciones puntuales en estado de reposo**. Para ello, se utilizará el módulo de instrumentación analógica AD8232 acoplado a un arreglo de tres electrodos de contacto seco (pernos M3), requiriendo que el usuario cierre el circuito bioeléctrico mediante contacto bimanual. Estos datos serán procesados inicialmente en el microcontrolador, para luego enviarse vía bluetooth al teléfono, el cual subirá los datos al servidor que guardará el perfil del usuario, junto con hacer un análisis más profundo de las métricas diarias.

4.4. Procesamiento algorítmico y Machine Learning

El dispositivo no solo actúa como un recolector de datos, sino que debe procesarlos localmente, cumpliendo con los siguientes requerimientos algorítmicos:

4.4.1. Contador de pasos algorítmico

Dadas las restricciones del proyecto de no utilizar módulos comerciales pre-programados, se desarrollará un algoritmo propietario en C. La Tabla 1 presenta una comparación de las principales familias de algoritmos de conteo de pasos reportados en la literatura para dispositivos wearable.

Algoritmo	Precisión	Costo comp.	Ventajas	Desventajas
Detección de picos con umbral adaptativo (Seleccionado)	92–96 %	Muy bajo	Simple, bajo consumo de RAM, robusto ante cambios de velocidad	Sensible a vibraciones; requiere filtrado previo
Cruce por cero (*Zero-Crossing*)	85–90 %	Muy bajo	Implementación trivial, sin parámetros de umbral	Alta tasa de falsos positivos con movimientos no periódicos
Autocorrelación temporal	93–97 %	Alto	Excelente con señales periódicas, estima frecuencia de paso	Costoso en CPU y RAM (requiere buffers grandes); no apto para tiempo real en C3
Clasificación por ML (CNN/LSTM)	95–98 %	Muy alto	Mayor precisión, distingue tipos de paso	Excesivo para un simple contador; ya se usa ML para clasificación de actividad

Cuadro 1: Comparación de algoritmos de conteo de pasos para dispositivos wearable.

Se seleccionó el algoritmo de **detección de picos con umbral adaptativo** por su excelente balance entre precisión (92–96 %) y costo computacional mínimo, siendo el más adecuado para la arquitectura RISC-V del ESP32-C3 [4] que carece de FPU por hardware. El pipeline de procesamiento opera sobre la señal del acelerómetro BMI160 [10] y consta de las siguientes etapas:

1. **Cálculo de magnitud:** Se computa $|a| = \sqrt{a_x^2 + a_y^2 + a_z^2}$ a partir de las lecturas del acelerómetro a 50 Hz, eliminando la dependencia de la orientación del dispositivo en la muñeca.
2. **Filtro pasabajos (Media Móvil):** Se aplica un filtro de media móvil de ventana $N = 10$ muestras (200 ms a 50 Hz), con el propósito de atenuar vibraciones de alta frecuencia y ruido mecánico. La frecuencia de corte resultante es $f_c \approx \frac{f_s}{2\pi N} \approx 0{,}8$ Hz, que retiene las componentes de paso humano típicas (1–3 Hz caminando, hasta 4 Hz corriendo).
3. **Detección de picos con umbral adaptativo:** Se implementa un detector de cruce por umbral con histéresis. El umbral se recalcula cada 5 segundos como el 60 % de la amplitud pico-a-pico observada en la ventana anterior. Un paso se contabiliza cuando la señal filtrada cruza el umbral ascendente seguido de un cruce descendente, con un período refractario mínimo de 250 ms entre detecciones consecutivas para evitar doble conteo.
4. **Validación temporal:** Se requiere un mínimo de 3 pasos consecutivos dentro de un intervalo de 5 segundos para confirmar que el usuario está caminando (evitando falsos positivos por movimientos aislados).

Este contador contará con la funcionalidad de ser reseteado a cero a voluntad del usuario navegando por los menús de la interfaz gráfica local del *smartwatch*.

4.4.2. Clasificación de actividades (4 estados)

Se requiere implementar un modelo de clasificación capaz de predecir cuatro estados físicos y situaciones de riesgo: *reposo*, *caminar*, *correr* y detección de *caída/emergencia*. Este último estado es crítico para otorgar un valor añadido a la seguridad del usuario.

La Tabla 2 presenta un análisis comparativo de arquitecturas de modelos de *Machine Learning* candidatos, basándose en benchmarks de HAR (*Human Activity Recognition*) en dispositivos edge [1].

Modelo	**Peso (KB)**	**Latencia (ms)**	**Precisión est.**	**Observaciones**
Decision Tree / Random Forest	2–10	<1	85–90 %	Bajo consumo de RAM. Interpretable. Limitado con patrones temporales complejos.
SVM (RBF Kernel)	5–20	1–5	88–92 %	Buen rendimiento con features manuales. Requiere extracción de características previa.
CNN 1D (Seleccionado)	15–50	5–15	92–96 %	Opera directamente sobre ventanas de aceleración cruda. Compatible con TFLite Micro y Edge Impulse.
LSTM / GRU	50–200	20–50	93–97 %	Excelente para series temporales, pero excesivo en memoria y latencia para ESP32-C3.

Cuadro 2: Comparación de arquitecturas de ML para clasificación de actividad en microcontroladores [1].

Se ha seleccionado una **Red Neuronal Convolucional 1D (CNN 1D)** como arquitectura base, compuesta por 2 capas convolucionales (8 y 16 filtros respectivamente, kernel de tamaño 3) seguidas de una capa de *Global Average Pooling* y una capa densa de 4 neuronas con activación *softmax*. Esta arquitectura ha demostrado un rendimiento superior en tareas de HAR sobre microcontroladores frente a modelos clásicos [1], y se entrenará utilizando **Edge Impulse** [12] con datos recolectados del propio BMI160 [10] montado en la muñeca, para luego exportar el modelo cuantizado a `int8` mediante **TensorFlow Lite for Microcontrollers** [13]. Los objetivos de diseño del modelo son:

- **Peso del modelo:** $\leq$ 30 KB (flash) y $\leq$ 10 KB (arena de inferencia en SRAM).
- **Latencia de inferencia:** $\leq$ 20 ms en ESP32-C3 a 160 MHz [4].
- **Precisión objetivo:** $\geq$ 90 % en las 4 clases, con énfasis en minimizar falsos negativos para la detección de caídas.

4.4.3. Justificación de la arquitectura Edge AI

Se ha decidido implementar el modelo de Inteligencia Artificial directamente en el microcontrolador (*Edge Computing/TinyML*) en lugar de procesarlo en la nube. Esta decisión se justifica por tres factores clave: minimiza la latencia (aspecto crítico para reportar una caída en tiempo real), reduce drásticamente el consumo energético al evitar la transmisión constante de telemetría cruda por la antena Bluetooth, y garantiza la privacidad de los movimientos del usuario.

4.5. Especificación de manejo de datos

Se detallan las especificaciones de los datos que el sistema genera, almacena y transmite. La Tabla 3 presenta el volumen, formato y destino de cada tipo de dato.

Tipo de dato	Tamaño	Frecuencia	Formato	Destino
Telemetría continua (HR, SpO2, Temp, Pasos, Actividad)	12 bytes	1 Hz (BLE Notify)	Struct empaquetado Little-Endian (uint8, int16, uint32)	Buffer local → BLE → Firestore
ECG crudo (AD8232)	2 bytes/muestra	100 Hz (30 s)	int16 (12-bit ADC en 16-bit container) [9]	BLE streaming → Cloud Functions
Ventana IMU (ML)	600 bytes	Cada 2 s	6 × int16 × 50 muestras [10]	Procesado local (TinyML)
Sesión (resumen)	∼200 bytes	Cada 5 min	JSON (Firestore document)	Cloud Firestore (persistencia) [14]
ECG procesado	∼500 bytes	Bajo demanda	JSON (BPM, HRV, R-peaks)	Cloud Functions → Firestore

Cuadro 3: Especificación de volumen, formato y destino de los datos del sistema.

La velocidad de transmisión BLE se configura a un *connection interval* de 15 ms con un MTU de 247 bytes [15], lo que permite un throughput efectivo de aproximadamente 13 kB/s. Este ancho de banda es suficiente para transmitir la telemetría continua (12 B/s) y los datos de ECG crudo (200 B/s a 100 Hz). El almacenamiento local en el microcontrolador utiliza un buffer circular de 4 KB en SRAM [4] para la telemetría, sin requerir almacenamiento persistente en flash, ya que los datos se transmiten en tiempo real.

4.6. Algoritmos de Procesamiento de Señales (ECG)

Para la estimación precisa de la frecuencia cardíaca a partir de la señal analógica del módulo AD8232, se implementará el algoritmo estándar de **Pan-Tompkins** [16]. Para no sobrecargar el microcontrolador y conservar batería, este procesamiento se ejecutará de manera asíncrona en la nube una vez recibida la trama de datos:

1. **Filtrado Digital Pasabanda:** Se aplicará un filtro digital para atenuar el ruido de alta frecuencia (interferencia de la red eléctrica) y eliminar la deriva de la línea base causada por la respiración del usuario.
2. **Procesamiento Morfológico:** La señal filtrada pasará por un filtro derivativo y una función de cuadratura para resaltar exclusivamente la pendiente aguda del complejo QRS del corazón.
3. **Detección de Picos R:** Se utilizará una integración de ventana móvil con umbrales adaptativos (*adaptive thresholding*) para identificar con precisión el tiempo exacto de cada latido.
4. **Cálculo de Métricas:** A partir de la distancia temporal entre picos consecutivos (intervalos R-R), el sistema calculará la Frecuencia Cardíaca promedio (BPM) y devolverá el valor a la aplicación móvil para su visualización.

4.7. Interfaz gráfica y navegación

Para asegurar una interacción intuitiva y cumplir con los requerimientos de visualización local sin depender exclusivamente del teléfono móvil, el dispositivo integrará una pantalla LCD rectangular a color de 1.69 pulgadas. El sistema de navegación físico constará de tres botones pulsadores (Arriba, Abajo y Seleccionar/Acción) ubicados en el costado derecho de la carcasa, garantizando una operación intuitiva y fácil, incluso durante la actividad física.

La interfaz gráfica de usuario (GUI) se ha diseñado maximizando el contraste mediante un fondo negro e íconos de colores vibrantes para una rápida lectura. El sistema se estructurará lógicamente en cuatro vistas principales, las cuales se pueden apreciar en la Fig.3:

Figura 3: Diseño preliminar de las interfaces gráficas del dispositivo.

1. **Pantalla Principal (Dashboard):** Actuará como la carátula central (*watchface*). Concentra la información de uso continuo exigida por los requerimientos: la hora actual, el contador de pasos, el nivel porcentual de batería, la frecuencia cardíaca y la actividad física predicha por el modelo de IA en ese instante.

2. **Biometría General:** Un panel dedicado a mostrar las lecturas de los sensores periféricos, desplegando la frecuencia cardíaca, el porcentaje de saturación de oxígeno (SpO2) y la temperatura corporal superficial, además de indicar el estado operativo de los sensores.

3. **Modo ECG:** Interfaz interactiva orientada exclusivamente a la toma del electrocardiograma en estado de reposo. Guiará al usuario mediante instrucciones visuales para que cierre el circuito bimanual tocando los electrodos. Adicionalmente, mostrará una cuenta regresiva del progreso y dibujará (se está evaluando la posibilidad) la forma de onda cardíaca en tiempo rea.

4. **Menú del Sistema:** Interfaz de configuración general que albergará opciones avanzadas. Aquí se incluirá la función de reiniciar a cero el contador de pasos, activar el modo de emparejamiento Bluetooth (BLE) para la sincronización de datos (*Data Logger*) con la aplicación móvil/PC y gestionar el apagado del equipo.

5. Elección de componentes y justificación

El componente más importante de nuestro *wearable* es el microcontrolador, ya que actúa como el cerebro del sistema: gobierna todos los periféricos, ejecuta los algoritmos de procesamiento de señales, corre el modelo de *Machine Learning* y gestiona las comunicaciones inalámbricas. Una elección inadecuada en esta etapa comprometería la autonomía energética, la capacidad de procesamiento y la viabilidad de futuras expansiones del producto.

A continuación se presenta una tabla comparativa de las distintas opciones analizadas como MCU del *wearable*. Es importante aclarar que los tamaños utilizados para la columna de dimensiones están referidos a módulos que tienen integrados los microcontroladores mencionados.

MCU	Arch y Freq	Math Process (DSP/ML)	Connect	Power Consumption	Size/ Integration	PinOut GPIO
ESP32-C3 SUPER MINI (Seleccionado) [4]	RISC-V 32-bit de un núcleo (160 MHz)	**Por software.** No posee FPU (hardware), los cálculos de ML y filtros requerirán más ciclos de reloj.	WiFi 4 y Bluetooth 5 (LE)	**Medio.** Requiere diseño cuidadoso de Deep Sleep.	22.5 mm x 18 mm	13
ESP32-S3 (Seleccionado) [5]	Xtensa 32-bit Dual-Core (240 MHz)	**Sobresaliente.** Posee instrucciones vectoriales para acelerar redes neuronales.	WiFi 4 y Bluetooth 5 (LE)	**Alto.** Consumo peak de 340 mA transmitiendo por Wifi 2.4 GHz, 1 Mbps y 21 dBm .	Muy bueno, pero requiere un área de PCB ligeramente mayor.	27
NRF52840 [17]	ARM Cortex-M4F (64 MHz)	**Excelente.** Posee FPU por hardware, ideal para el filtrado del ECG y ML de bajo consumo.	Solo Bluetooth 5 (LE) y otros. **Sin WiFi.**	**Ultra Bajo.** Perfecto para wearables de alta autonomía.	Excelente, existen módulos SMD diminutos.	11
STM32WB55 [18]	Dual-Core: ARM Cortex-M4 (64 MHz) para aplicación y Cortex-M0+ (32 MHz) dedicado a radio.	**Excelente.** Posee FPU por hardware y soporte nativo optimizado para ML	Bluetooth 5 (LE) y 802.15.4 (Zigbee/Thread). **Sin WiFi.**	**Ultra Bajo.** El procesador de radio independiente permite ahorrar mucha batería durante las transmisiones.	**Muy bueno.** Empaquetados muy pequeños, pero requiere un diseño de antena RF si no se compra en formato módulo.	22

Cuadro 4: Comparación de Opciones de Microcontroladores (MCU)

La decisión de utilizar la familia de microcontroladores de Espressif (**ESP32-C3** y **ESP32-S3**) se fundamenta en su **facilidad de desarrollo** y la **vasta cantidad de documentación** y herramientas disponibles que el equipo ya domina. Esto permite acelerar significativamente la fase de prototipado. Adicionalmente, la compatibilidad en el ecosistema de Espressif ofrece una tremenda flexibilidad: el proyecto iniciará utilizando el **ESP32-C3** debido a su bajo costo y menor consumo energético, con la **posibilidad de migrar fácilmente al ESP32-S3** sin mayores cambios en el código base, en caso de que el modelo

de *Machine Learning* y los algoritmos de procesamiento exijan una **mayor potencia de cómputo**.

Sensor (Fabricante)	**Interfaz**	**Resolución**	**Consumo (Modo Activo)**	**Ventajas / Desventajas**
BMI160 (Bosch) - Seleccionado [10]	I2C / SPI	16 bits (Acel + Giro)	∼900 μA	**Ventaja:** Excelente eficiencia y buffer FIFO grande (1024 bytes). Ideal para Edge ML.
MPU6050 (InvenSense) [19]	I2C	16 bits (Acel + Giro)	∼3900 μA (3.9 mA)	**Desventaja:** Consumo de energía excesivamente alto para un wearable. Tecnología antigua (Legacy).
LSM6DS3 (STMicro) [20]	I2C / SPI	16 bits (Acel + Giro)	∼1250 μA	**Ventaja:** Muy bajo consumo y contador de pasos por hardware integrado (aunque el proyecto exige software propio).

Cuadro 5: Comparación de Sensores Inerciales (IMU) para Detección de Actividad

Para la detección de movimiento continuo (podómetro y clasificación de Machine Learning), la Tabla 5 evidencia que el BMI160 es la opción idónea al combinar un consumo por debajo de 1 mA y un amplio buffer FIFO (que desacopla el trabajo de lectura continua del microcontrolador), descartando alternativas de tecnología más antigua y alto perfil energético como el MPU6050.

Sensor (Fabricante)	Interfaz	Resolución ADC	Consumo Promedio	Ventajas / Desventajas
MAX30102 (Maxim) - Seleccionado [7]	I2C	18 bits	~600 μA	**Ventaja:** Integra LEDs y fotodetector. Alta resolución y cristal protector que facilita la limpieza en contacto con la piel.
MAX30100 (Maxim) [21]	I2C	16 bits	~1200 μA	**Desventaja:** Menor resolución, mayor consumo y conocido por problemas de compatibilidad de voltajes en módulos comerciales.
MAX86141 (Maxim) [22]	SPI	19 bits	Ultra bajo (<100 μA)	**Desventaja:** Requiere LEDs externos y diseño de hardware sumamente complejo. Excede el alcance de un prototipo inicial.

Cuadro 6: Comparación de Sensores Ópticos (PPG) para HR y SpO2

Por el lado de la métrica cardíaca y de oxigenación, en la Tabla 6 se analiza el abanico de fotopletismógrafos comerciales. El MAX30102 se consagra como nuestra elección principal por su excelente relación de facilidad de uso e integración, ya que compila ópticas de alta resolución en un encapsulado que incluye vidrio protector ideal para el contacto epitelial constante en la muñeca, superando las limitaciones de versiones antiguas como el MAX30100.

Sensor (Fabricante)	**Interfaz**	**Precisión (Rango Clínico)**	**Consumo Activo**	**Ventajas / Desventajas**
MAX30205 (Maxim) - Seleccionado [8]	I2C	±0.1°C (37°C a 39°C)	∼600 μA	**Ventaja:** Precisión de grado clínico específica para piel humana. Fácil integración en bus I2C.
MLX90614 (Melexis) [23]	I2C / PWM	±0.2°C	∼1.5 mA	**Desventaja:** Medición infrarroja sin contacto. Es muy voluminoso (empaque TO-39) e inviable para un diseño ultracompacto.
DS18B20 (Maxim) [24]	1-Wire	±0.5°C	∼1 mA	**Desventaja:** Precisión insuficiente para uso biomédico y el protocolo 1-Wire consume más recursos de software que el I2C.

Cuadro 7: Comparación de Sensores de Temperatura para Aplicaciones Médicas/Wearables

La medición de fiebre o estrés térmico requiere alta fidelidad; como describe la Tabla 7, descartamos medidores de propósito general (DS18B20) o módulos infrarrojos muy abultados (MLX90614, propio de termómetros de pistola). En contraste, el MAX30205 proporciona precisión con certificación clínica (±0.1°C en rango humano) mediante simple contacto superficial con la piel.

IC de Carga (Fabricante)	Topología	Protección Integrada	Corriente de Carga	Ventajas / Desventajas
TP4056 + DW01A - Seleccionado [25]	Cargador Lineal LDO	Requiere IC externo (DW01A)	Hasta 1A (Ajustable)	**Ventaja:** Costo extremadamente bajo, circuito comprobado y fácil de soldar a mano. Ideal para el prototipo.
BQ24075 (Texas Instruments) [26]	Cargador Lineal con Power-Path	Sí (Sobretensión y Temperatura)	Hasta 1.5A	**Desventaja:** Costoso y empaquetado QFN difícil de ensamblar. **Ventaja:** Permite usar el dispositivo mientras se carga sin estresar la batería.
MCP73831 (Microchip) [27]	Cargador Lineal	No (Solo gestión térmica)	Hasta 500 mA	**Desventaja:** Aunque es diminuto (SOT-23), carece de protección contra sobredescarga, obligando a añadir más circuitería.

Cuadro 8: Comparación de Circuitos Integrados para Carga y Protección (BMS)

Respecto a la fuente de poder, se seleccionó una **batería de polímero de litio (LiPo) de 3.7V**. Para justificar la capacidad seleccionada, la Tabla 9 presenta un desglose detallado del consumo energético estimado de cada subsistema, basándose en los valores típicos de las hojas de datos de cada componente.

Componente	Modo Activo (mA)	Modo Reposo (mA)	Ciclo de uso (%)
ESP32-C3 (CPU + Radio off) [4]	15.0	0.05	100
ESP32-C3 (BLE Tx, periódico) [4]	5.0	0.0	10
Pantalla ST7789 (backlight) [11]	25.0	0.0	40
MAX30102 (PPG activo) [7]	0.6	0.0	50
MAX30205 (Temperatura) [8]	0.6	0.01	5
BMI160 (IMU normal) [10]	0.9	0.003	100
MAX17048 (Fuel Gauge) [28]	0.05	0.05	100
AD8232 (ECG, bajo demanda) [9]	0.17	0.0	2
LDO ME6211 (I_Q) [29]	0.055	0.055	100
Total ponderado	**∼22.9**	**∼0.17**	—

Cuadro 9: Desglose del consumo energético estimado por componente (valores de hojas de datos).

Considerando un perfil de uso diario de **8 horas de modo activo** (pantalla encendida, sensores operando) y **16 horas de modo reposo** (pantalla apagada, solo IMU activo), el consumo promedio ponderado resulta:

$$I_{avg} = \frac{8 \times 22{,}9 + 16 \times 0{,}17}{24} \approx 7{,}7 \text{ mA} \tag{1}$$

Con una batería de **250 mAh** (modelo 502030, dimensiones 30×20×5 mm, con circuito de protección PCM integrado) y considerando una eficiencia del LDO del 90 % [29], la autonomía teórica es:

$$t_{autonomia} = \frac{250 \times 0{,}9}{7{,}7} \approx 29{,}2 \text{ horas} \approx 1{,}2 \text{ días} \tag{2}$$

Este cálculo valida la selección de la celda de 250 mAh, superando holgadamente el requerimiento mínimo de 12 horas de operación.

Consideraciones mecánicas: El diseño contempla una arquitectura tipo *sandwich* de doble PCB (PCB superior con MCU y conector de pantalla, batería en el centro, PCB inferior con los sensores de contacto con la piel: MAX30102 y MAX30205). Se utilizará la misma celda 502030 en ambas versiones del producto. La Tabla 10 presenta el análisis de apilamiento (*stack-up*).

Capa	Prototipo (mm)	Versión Pro (mm)
Display + cristal	2.5	2.0
Top PCB (MCU, conectores)	1.6	1.0
Batería LiPo 502030	5.0 (250 mAh)	5.0 (250 mAh)
Bottom PCB (sensores skin-contact)	1.6	1.0
Cover posterior (ventana óptica PPG)	0.5	0.5
Carcasa (paredes superior e inferior)	1.5	1.2
Espesor total	**∼12.7**	**∼10.7**
Referencia: Apple Watch (10.7), Galaxy Watch (9.8)	*Rango comercial: 9–11 mm*	

Cuadro 10: Análisis de apilamiento mecánico (*stack-up*) para la arquitectura sandwich de doble PCB.

La huella de 20×30 mm de la celda 502030 se ajusta al espacio disponible bajo la pantalla de 1.69" (∼35×42 mm), dejando margen para el ruteo de cables y conectores FPC. La versión "Pro" (PCB SMD de 4 capas fabricada en JLCPCB) reduce el espesor total a ∼10.7 mm gracias a PCBs de 1.0 mm y una carcasa más delgada, igualando el perfil de un Apple Watch.

Para la gestión térmica y energética de dicha celda, la Tabla 8 resume el análisis del subsistema de carga (BMS). La combinación de los integrados genéricos TP4056 y el circuito de protección DW01A es ideal para el primer prototipo. Este módulo se adquiere fácilmente soldado en placa base con puerto Type-C integrado por un valor bajo, superando las barreras de soldadura que ofrecen QFNs como el BQ24075.

Componente (Fabricante)	Método de Medición	Hardware Externo	Consumo Promedio	Ventajas / Desventajas
MAX17043 (Maxim) - Seleccionado [30]	ModelGauge (Basado en voltaje)	Ninguno	~50 μA	**Ventaja:** Estima el estado de carga (SOC) sin resistencia de sensado (shunt), ahorrando valioso espacio en la PCB.
BQ27441-G1 (Texas Instruments) [31]	Impedance Track (Coulomb Counting)	Resistencia Shunt externa	~93 μA	**Desventaja:** Mayor precisión, pero requiere ruteo crítico de la resistencia shunt y configuración compleja del perfil de batería.
Lectura directa por ADC (ESP32)	Divisor de voltaje resistivo	2 Resistencias	Variable (Corriente de fuga)	**Desventaja:** El ADC del ESP32 no es lineal. Desperdicia batería por el divisor de tensión y es muy impreciso.

Cuadro 11: Comparación de Monitores de Batería (Fuel Gauges)

Representar el *State of Charge* (nivel de batería en la pantalla de la muñeca) sin dañar la autonomía general requiere un integrado especializado. Tal como indica la Tabla 11, la decisión decantó por el MAX17043 ya que utiliza el algoritmo patentado *ModelGauge* sobre el voltaje crudo (sin utilizar derivadores shunt externos o puentes resistivos analógicos), ahorrando considerable consumo estático en la placa.

Regulador LDO	Corriente Máxima	Voltaje de Caída (Dropout)	Corriente Quiescente (I_Q)	Ventajas / Desventajas
ME6211 / AP2112K - Seleccionado [29]	∼500 mA	Ultra bajo (∼100 mV a 100 mA)	∼55 μA	**Ventaja:** Excelente balance entre tamaño (SOT-23-5), eficiencia cuando la batería baja de 3.6V y capacidad de corriente para el WiFi.
AMS1117-3.3 [32]	1 A	Muy alto (∼1.1 V)	∼5 mA (5000 μA)	**Desventaja:** Pésima elección para baterías. Su alto dropout cortaría el ESP32 incluso cuando la batería esté al 50 %. Drena energía en reposo.
TPS7A02 (Texas Instruments) [33]	200 mA	Bajo (∼140 mV)	Nano-poder (25 nA)	**Desventaja:** Corriente máxima insuficiente para los picos de transmisión WiFi del ESP32-C3 (que superan los 300 mA).

Cuadro 12: Comparación de Reguladores de Voltaje LDO (3.3V) para Fase 2

Traducir los voltajes inestables de la batería de litio (4.2V a 3.0V) a los estables 3.3V que demanda el microcontrolador debe hacerse con máxima eficiencia térmica y con el mínimo dropout posible (Voltaje de caída extra). La Tabla 12 deja en evidencia que reguladores comunes (como el AMS1117) son pésimos para baterías móviles; por ello, elegimos el ME6211, el cual puede entregar ráfagas de alta corriente sin bloquearse cuando la celda baje de 3.6V.

Tecnología (Controlador)	Interfaz	Color y Resolución	Consumo Estimado	Ventajas / Desventajas
IPS LCD 1.3"(ST7789) - Seleccionado [11]	SPI	RGB (240x240)	Medio (∼20-40 mA con retroiluminación)	**Ventaja:** Excelente densidad de píxeles, colores vibrantes para gráficas (ECG) y la interfaz SPI maneja la tasa de refresco sin cuellos de botella.
OLED 0.96"(SSD1306) [34]	I2C / SPI	Monocromo (128x64)	Bajo-Medio (∼10-20 mA)	**Desventaja:** Menor resolución y tamaño. Difícil visualizar 4 señales simultáneas y la predicción de Machine Learning de forma legible.
Memory LCD (Sharp) [35]	SPI	Monocromo (Alto contraste)	Ultra bajo (∼10 μA)	**Desventaja:** Muy costosa y sin retroiluminación (ilegible en la oscuridad). Excelente para autonomía extrema, pero no justifica el costo en este alcance.

Cuadro 13: Comparación de Tecnologías de Pantalla para Interfaz de Usuario

Finalmente, dotar al sistema de una cara visual local y dinámica decanta en la elección del controlador de pantalla ST7789 (Tabla 13). Al descartar opciones monocromáticas limitadas o extremadamente costosas, la pantalla IPS permite un pintado en alta resolución que es ideal para dibujar los vectores del Electrocardiograma (ECG) en vivo, junto a una paleta de colores vibrantes para un entorno de usuario premium.

6. Diagrama de bloques

6.1. Diagrama de bloques de alto nivel

Para la implementación de la propuesta, se planteó el diagrama de bloques de la Fig.4, el cual presenta una vista integral de todos los subsistemas del proyecto SupaClock y sus interacciones. El diagrama abarca desde la adquisición de señales biométricas en el dispositivo hasta el almacenamiento y análisis de datos en la nube, pasando por la aplicación móvil que actúa como enlace entre ambos dominios. Los colores de las cajas y las conexiones permiten identificar rápidamente cada dominio funcional, tal como se detalla en la leyenda incluida en la parte inferior del diagrama.

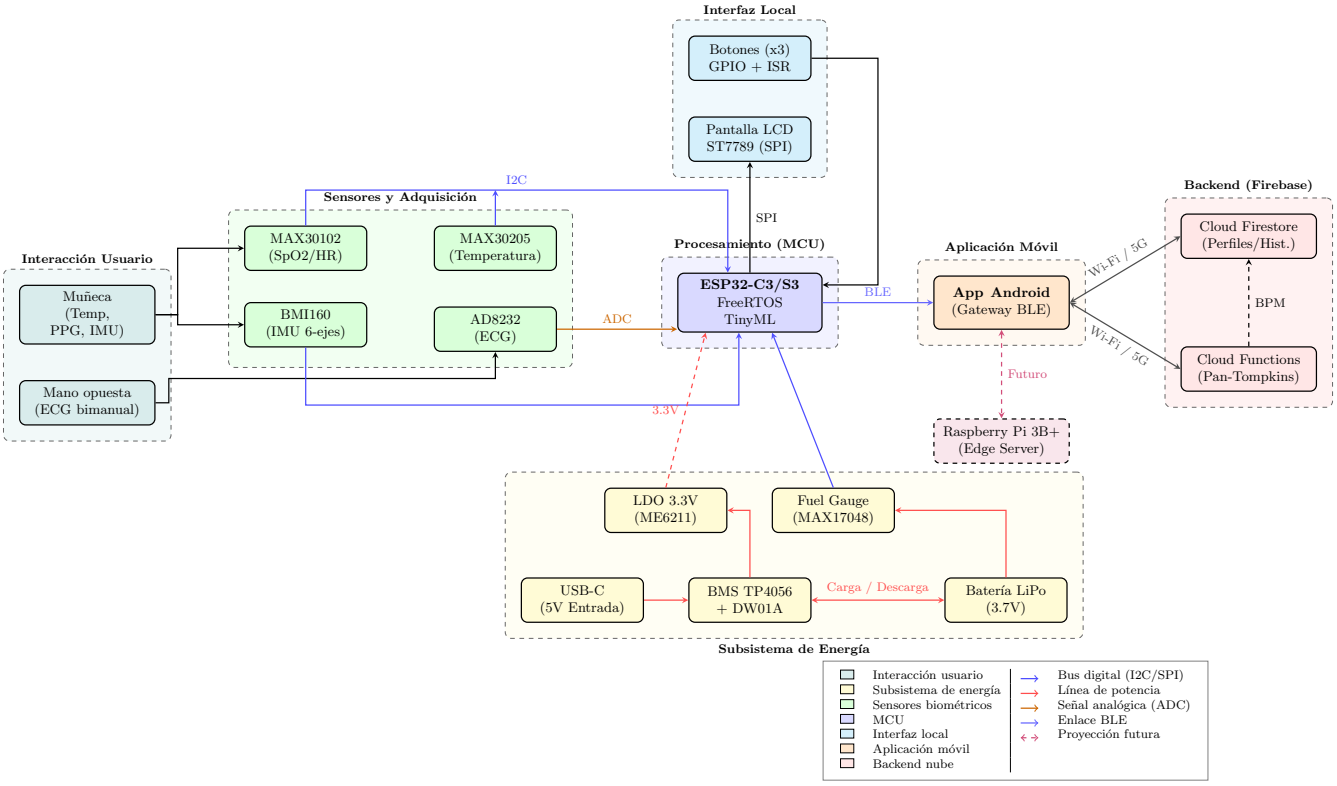

Figura 4: Diagrama de bloques de alto nivel del sistema SupaClock, incluyendo el dispositivo, la aplicación móvil y el backend en la nube.

1. **Bloque de Interacción (Usuario):** Representa el vínculo físico entre el wearable y el individuo. Diferencia la zona de la muñeca (para la captura constante de temperatura, inercia y fotopletismografía y ECG) de la extremidad opuesta (necesaria para cerrar el circuito bioeléctrico del ECG bimanual). A su vez, ilustra la retroalimentación visual y táctil mediante la pantalla y los botones.

2. **Bloque de Energía:** Está compuesto por una celda de polímero de litio (LiPo), un módulo BMS TP4056 que garantiza una recarga segura por USB-C, y un regulador lineal (LDO) que estabiliza el voltaje a una pista principal de 3.3V, alimentando a toda la electrónica de forma ininterrumpida.

3. **Bloque de Sensores y Adquisición:** Constituye la etapa de instrumentación periférica. Agrupa al sensor MAX17048 (para telemetría de batería), MAX30102 (SpO2/HR), MAX30205 (Temperatura) y BMI160 (IMU), los cuales multiplexan su información digital a través de un bus I2C unificado. De forma paralela, el módulo AD8232 entrega una señal analógica continua para el electrocardiograma.

4. **Bloque de Procesamiento (MCU):** El núcleo del dispositivo, operado por el microcontrolador ESP32-C3 bajo el sistema en tiempo real FreeRTOS. Se encarga de gobernar los periféricos, ejecutar el filtrado digital local y correr el modelo de *Machine Learning* (*TinyML*) para la inferencia y clasificación de actividades del usuario.

5. **Bloque de Interfaz Local:** Permite la operatividad local sin depender del teléfono móvil. Está compuesto por un panel LCD IPS (ST7789) manejado mediante un bus de alta velocidad SPI (vía acceso directo a memoria, DMA) y un set de botones configurados como interrupciones por hardware (GPIO).

6. **Bloque de Aplicación Móvil:** La aplicación Android actúa como *Gateway* entre el dispositivo y la nube. Recibe datos vía BLE, los visualiza en tiempo real y los reenvía al backend para su almacenamiento y análisis.

7. **Bloque de Backend (Firebase):** Cloud Firestore almacena perfiles y métricas históricas, mientras

que Cloud Functions ejecuta algoritmos de procesamiento pesado (como Pan-Tompkins para ECG). Como proyección futura, se contempla una Raspberry Pi 3B+ como servidor local.

6.2. Diagrama de potencia

El subsistema de energización y distribución de potencia ha sido diseñado para garantizar la seguridad operativa de la batería y proveer un voltaje altamente estable a la electrónica sensible. El flujo de energía se estructura en tres etapas principales, tal como se ilustra en la Fig.5:

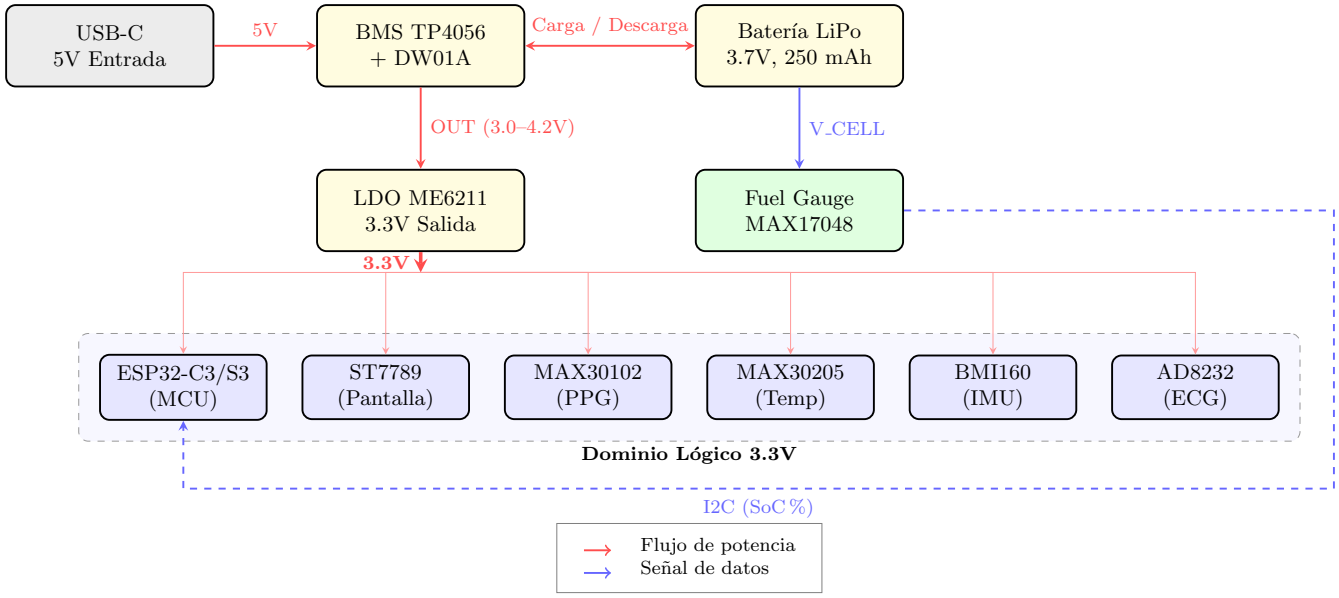

Figura 5: Diagrama de flujo de potencia y gestión de batería del sistema SupaClock.

- **Fuentes de Entrada y Gestión (BMS):** El sistema posee un funcionamiento autónomo alimentado por una batería LiPo de 3.7V. Para el proceso de recarga, se utiliza un puerto USB tipo C que suministra 5V a los pines de entrada (IN+/IN-) del módulo de gestión de batería (BMS) TP4056. Este módulo actúa como intermediario, encargándose de cargar la celda de forma segura (pines B+/B-) y aislar el circuito para protegerlo contra sobredescargas y cortocircuitos mediante el integrado DW01A.
- **Monitoreo Independiente (Fuel Gauge):** Para informar el nivel de carga al usuario, se ha integrado el sensor MAX17048. Este componente realiza una lectura de voltaje directa desde los terminales de la celda LiPo, operando en paralelo al BMS.
- **Regulación y Dominio Lógico (3.3V):** La salida de energía protegida (OUT+/OUT-) proveniente del BMS se direcciona hacia un regulador lineal (LDO). Este componente estabiliza la tensión fluctuante de la batería y establece una pista principal constante de 3.3V. Este dominio lógico unificado alimenta simultáneamente a todos los circuitos integrados del sistema: el microcontrolador central ESP32-C3 (S3 en posteriores versiones), la pantalla IPS ST7789, el acelerómetro BMI160, los biosensores MAX30102 y MAX30205, y el acondicionador de señal analógico del ECG (AD8232).

6.3. Diagrama de señales y buses de comunicación

La arquitectura de comunicación del *SupaClock* ha sido diseñada para minimizar el uso de pines del microcontrolador, agrupar sensores de la misma naturaleza y facilitar la futura migración del ESP32-C3

al ESP32-S3 de ser necesario (más que nada por temás de potencia a la hora de probar el *Machine Learning*). El flujo de datos se ilustra en la Fig.6:

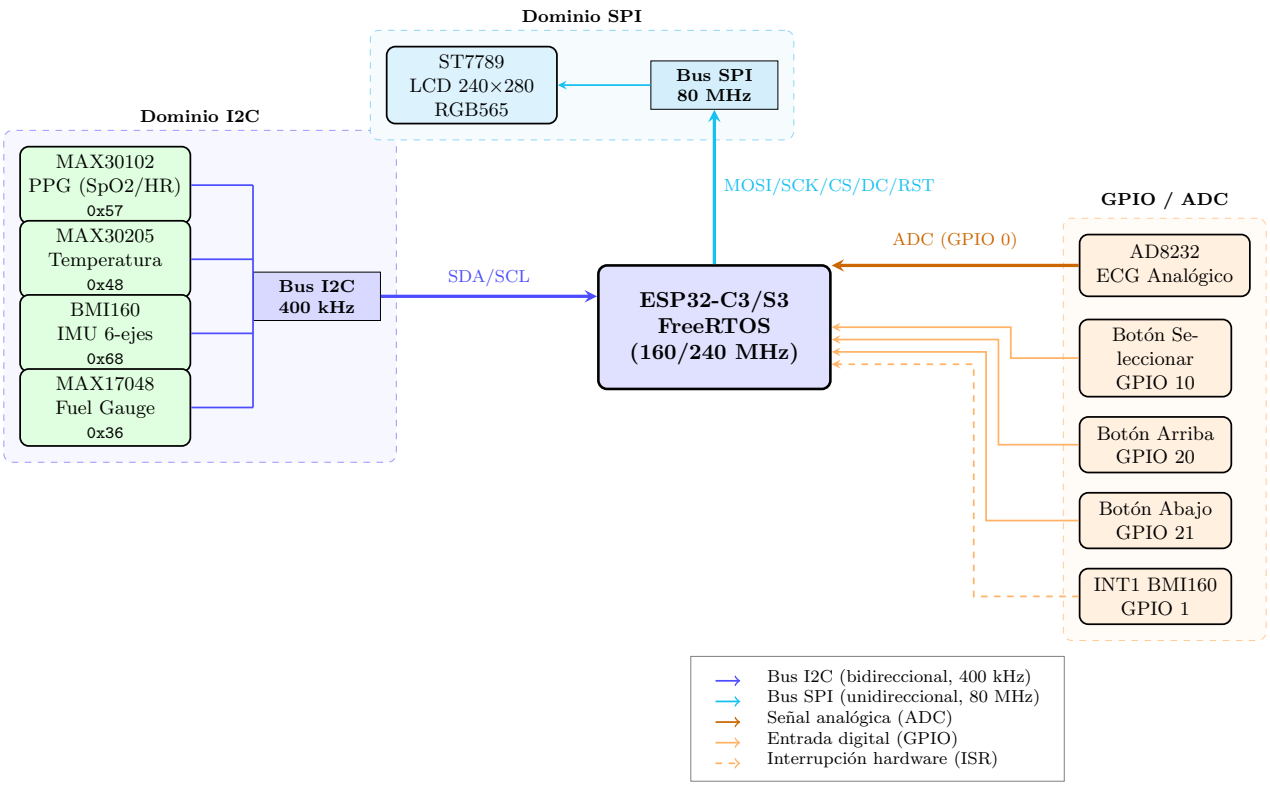

Figura 6: Diagrama de arquitectura de señales, buses lógicos y direcciones I2C.

El sistema divide las señales en cuatro dominios principales:

- **Bus I2C (Sensores):** Actúa como el bus principal de telemetría operando a 400 kHz. Agrupa el sensor de oximetría (MAX30102 - `0x57`), el monitor de batería (MAX17048 - `0x36`), el sensor de temperatura (MAX30205 - `0x48`) y la unidad inercial (BMI160 - `0x68`). El uso de este bus compartido es crítico para ahorrar puertos de I/O.
- **Bus SPI (Pantalla):** Un bus dedicado de alta velocidad. Está asignado al controlador ST7789 de la pantalla LCD. Solo requiere señales de salida (MOSI, Reloj, CS, DC, Reset), ya que no es necesario leer información desde la pantalla, ahorrando el pin MISO.
- **Canal Analógico (ECG):** La salida analógica filtrada proveniente del módulo AD8232 requiere de un conversor analógo-digital, el cual funcionará mediante la DMA. Este se conectará a un pin ADC dedicado.
- **Entradas Digitales (Botones e Interrupciones):** Tres líneas GPIO configuradas como entradas con resistencias *pull-up* internas para los botones físicos, y un pin adicional conectado al pin `INT1` del BMI160, permitiendo despertar al microcontrolador mediante este mismo de ser necesario a futuro.

6.3.1. Mapeo de pines y plan de migración (C3 a S3)

El proyecto contempla el desarrollo de los primeros prototipos con la placa ESP32-C3 *SuperMini* (que posee 13 GPIOs expuestos) y la migración final hacia el ESP32-S3 *SuperMini* (que posee 16 GPIOs), esto es altamente probable debido a los requerimientos de *Machine Learning*, pero debido al bajo costo monetario

de los módulos, preferimos probar primero con el de menor consumo. Tras analizar los esquemáticos de ambas placas, se confirma que la cantidad de pines es suficiente. Aprovechando la matriz GPIO, se ha definido un mapa de pines (Tabla 14) que intenta mantener la compatibilidad física entre ambas arquitecturas:

Periférico / Señal	**Pin ESP32-C3**	**Pin ESP32-S3**	**Tipo de Señal**
Bus I2C (SDA)	GPIO 8	GPIO 33	Digital Bidireccional
Bus I2C (SCL)	GPIO 9	GPIO 34	Digital Reloj
Pantalla SPI (MOSI)	GPIO 6	GPIO 6	Digital Salida
Pantalla SPI (SCK)	GPIO 4	GPIO 4	Digital Reloj
Pantalla SPI (CS)	GPIO 5	GPIO 5	Digital Salida
Pantalla SPI (DC)	GPIO 3	GPIO 3	Digital Salida
Pantalla SPI (RST)	GPIO 7	GPIO 7	Digital Salida
ECG (AD8232 Out)	GPIO 0 (A0)	GPIO 1	Analógica (ADC)
Interrupción IMU (INT1)	GPIO 1	GPIO 2	Entrada Digital
Botón 1 (Seleccionar)	GPIO 10	GPIO 14	Entrada Digital (Pull-up)
Botón 2 (Arriba)	GPIO 20	GPIO 15	Entrada Digital (Pull-up)
Botón 3 (Abajo)	GPIO 21	GPIO 16	Entrada Digital (Pull-up)

Cuadro 14: Mapeo definitivo de señales para placas SuperMini.

6.4. Arquitectura de Firmware (FreeRTOS)

Para gestionar la concurrencia de múltiples sensores y garantizar tiempos de ejecución deterministas, el microcontrolador ESP32 operará bajo el sistema operativo en tiempo real FreeRTOS. La arquitectura interna del dispositivo se estructurará de la siguiente manera:

- **Gestión de Tareas (Multitasking):** El sistema se dividirá en tareas independientes asignadas a distintos niveles de prioridad para optimizar el uso del procesador:
  - *Tarea de Adquisición (Prioridad Alta):* Encargada de muestrear el ADC (para el ECG) mediante acceso directo a memoria (DMA) y leer los buses I2C a frecuencias precisas.
  - *Tarea de Procesamiento Edge (Prioridad Media):* Ejecutará el modelo de *TinyML* para la clasificación de actividad y el algoritmo del contador de pasos de forma local.
  - *Tarea de Interfaz Gráfica (Prioridad Media-Baja):* Actualizará la pantalla LCD vía SPI y gestionará las interrupciones de los botones físicos para la navegación fluida.
  - *Tarea de Comunicación BLE (Prioridad Baja):* Empaquetará los datos y manejará la conexión con el teléfono móvil de forma asíncrona, evitando bloquear el sistema principal.
- **Sincronización y Comunicación (IPC):** Para evitar colisiones en los buses de datos (como el bus I2C compartido por cuatro sensores biométricos), se implementarán semáforos de exclusión mutua (*Mutexes*). La transferencia de datos entre la tarea de adquisición y la de procesamiento se realizará mediante colas de mensajes (*Queues*), asegurando que no se pierdan muestras críticas.
- **Gestión Energética Activa:** El planificador de FreeRTOS se configurará en conjunto con las rutinas de bajo consumo del ESP32 (*Light Sleep*). El sistema dormirá la mayor parte del tiempo y será despertado únicamente por interrupciones de hardware o temporizadores internos.

6.4.1. Frecuencias de Muestreo y Optimización de Memoria

Para optimizar el uso de la memoria SRAM del microcontrolador y el ancho de banda de transmisión Bluetooth, se diferenciarán las tasas de muestreo interno (*hardware*) de las tasas de registro lógico:

- **Sensores Dinámicos (IMU y PPG):** Muestreados internamente a frecuencias elevadas (50 a 100 Hz) para alimentar el modelo de *Machine Learning* y algoritmos de biometría. Los datos crudos no serán almacenados; el microcontrolador extraerá las características en tiempo real y solo registrará en el *buffer* de transmisión los resultados procesados a 1 Hz.
- **Sensores de Dinámica Lenta (Temperatura):** El sensor operará a 1 Hz, dada la alta inercia térmica del cuerpo humano.
- **Telemetría Cruda (ECG):** Durante la medición puntual, el ADC operará a 100 Hz mínimo. Estos datos sí serán almacenados de forma cruda (*raw*) y transmitidos inmediatamente por BLE, delegando su procesamiento a la nube.

La Fig.7 presenta el diagrama de bloques del firmware, mostrando la arquitectura de tareas concurrentes bajo FreeRTOS, los periféricos de hardware conectados al microcontrolador y los mecanismos de comunicación entre procesos (IPC). Las cajas grises representan los periféricos de hardware, las cajas azules las tareas de software, las cajas amarillas las colas de mensajes (*Queues*) y el círculo rojo el semáforo de exclusión mutua (*Mutex*) que protege el bus I2C compartido. Las flechas continuas indican flujo de datos y las discontinuas representan interrupciones asíncronas.

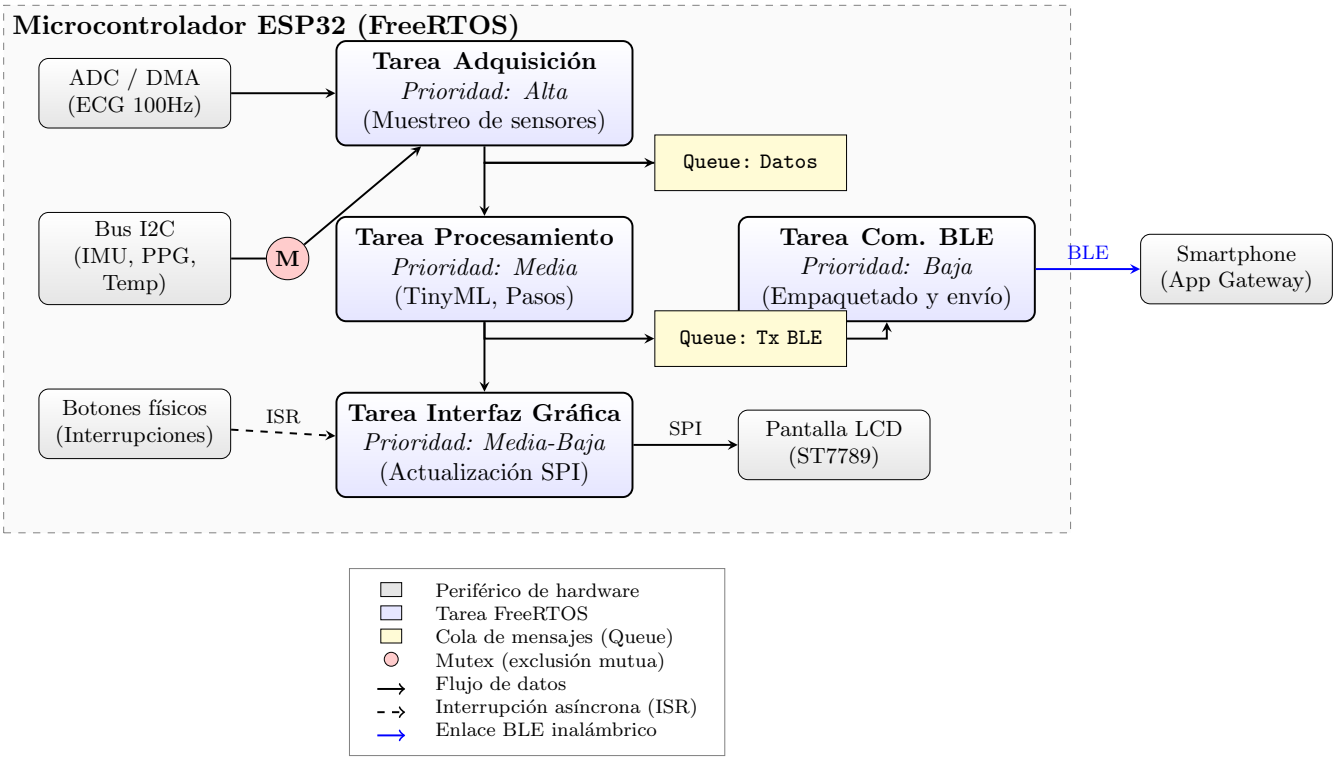

Figura 7: Diagrama de bloques del firmware y gestión de tareas en FreeRTOS.

6.5. Arquitectura de Ecosistema (App y Servidor)

El dispositivo actuará como un nodo de adquisición en el borde (*Edge*), delegando el almacenamiento histórico y el análisis profundo a una arquitectura externa compuesta por una Aplicación Móvil y un backend en la nube.

- **Estrategia de Transmisión (BLE):** Para optimizar el consumo energético, la transmisión Bluetooth Low Energy (BLE) operará en dos modalidades. La telemetría continua (pasos, temperatura, SpO2, actividad) se enviará a la aplicación móvil en paquetes consolidados de forma periódica. Para la medición del electrocardiograma, se abrirá un canal de alto rendimiento que transmitirá los datos a 100 Hz en tiempo real durante los 30 segundos de captura.
- **Aplicación Móvil (Android):** Funcionará como el enlace principal o *Gateway*. Se encargará de emparejarse con el dispositivo, visualizar el resumen en tiempo real, calibrar métricas del usuario y subir los paquetes de datos al servidor.
- **Backend Principal (Firebase):** Durante las primeras fases de desarrollo, se utilizará el ecosistema de Firebase. *Cloud Firestore* almacenará los perfiles y métricas históricas, mientras que el procesamiento pesado se ejecutará mediante *Cloud Functions* utilizando Python. Esto asegura alta disponibilidad y un despliegue rápido.
- **Proyección a Servidor Propio (Edge Server):** Como objetivo extendido, y sujeto a la holgura en la Carta Gantt, se contempla la migración del backend a un servidor propio gestionado mediante una Raspberry Pi 3B+. Esta arquitectura *On-Premise* (utilizando una API propia, bases de datos de series de tiempo e interfaces como Grafana) eliminaría la dependencia de servicios externos y maximizaría la privacidad de los datos biométricos.

La Fig.8 ilustra esta arquitectura de ecosistema. En el diagrama, las flechas azules representan la telemetría continua enviada a baja frecuencia (1 Hz), mientras que las flechas rojas indican el canal de alta velocidad para la transmisión de datos crudos de ECG (100 Hz). Las cajas verdes corresponden a dispositivos físicos, las azules a la aplicación de software y las naranjas al ecosistema en la nube. El nodo punteado en púrpura representa la proyección futura del servidor local.

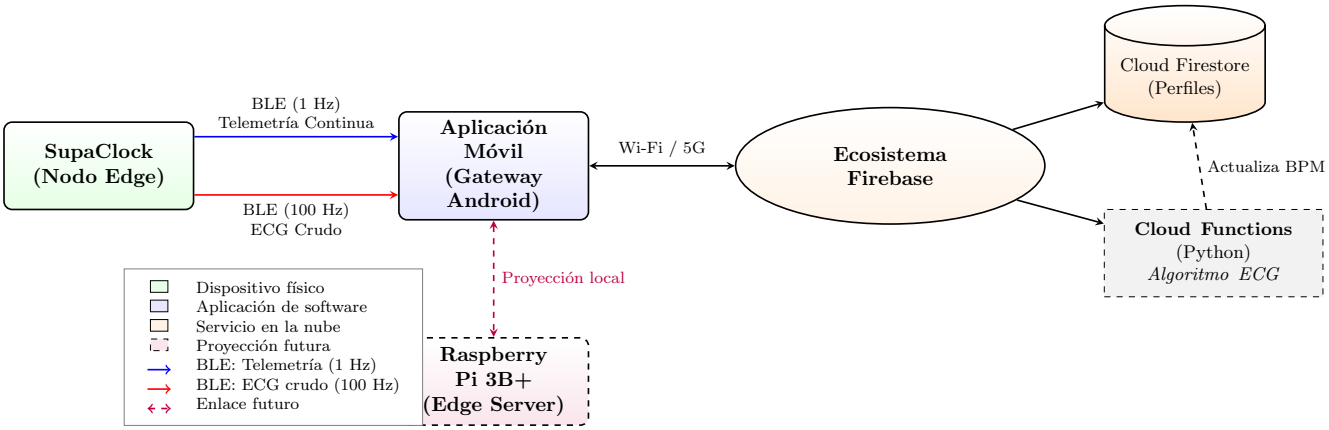

Figura 8: Arquitectura de ecosistema: flujo de datos desde el dispositivo *Edge* hacia la aplicación móvil y el backend en la nube.

6.5.1. Algoritmos de Procesamiento de Señales (ECG en Servidor)

Para la estimación de la frecuencia cardíaca a partir de la señal del AD8232, se implementará el algoritmo de **Pan-Tompkins** de manera asíncrona en el servidor (ej. funciones nativas en Python):

1. **Filtrado Digital Pasabanda:** Atenuación del ruido de alta frecuencia (interferencia electromagnética) y eliminación de la deriva de la línea base (*baseline wander*).
2. **Procesamiento Morfológico:** Aplicación de un filtro derivativo y función de cuadratura para resaltar la pendiente aguda del complejo QRS.

3. **Detección de Picos R:** Integración de ventana móvil combinada con umbrales adaptativos (*adaptive thresholding*) para identificar el instante exacto de cada latido.
4. **Cálculo de Métricas:** Evaluación de la distancia temporal entre picos consecutivos (intervalos R-R) para calcular los latidos por minuto (BPM) y evaluar la Variabilidad de la Frecuencia Cardíaca (HRV).

7. Bosquejo de la solución

La Fig.9 presenta el bosquejo preliminar del prototipo SupaClock, el cual ilustra las principales vistas y dimensiones del dispositivo. Desde la vista cenital se aprecia la interfaz del usuario con la pantalla principal y los íconos de navegación. En la vista frontal se observan los tres botones físicos ubicados en el costado derecho, los cuales cumplirán las funciones de navegación (arriba/abajo) y selección/acción. La vista lateral derecha muestra la ubicación de los electrodos de contacto seco para la medición de ECG, posicionados en la zona más cercana a la mano del usuario. Finalmente, la vista posterior exhibe el puerto USB-C de carga y la correa ajustable. Las cotas indicadas corresponden a las dimensiones máximas de diseño: largo $\leq$ 80 mm, ancho $\leq$ 50 mm y altura $\leq$ 30 mm.

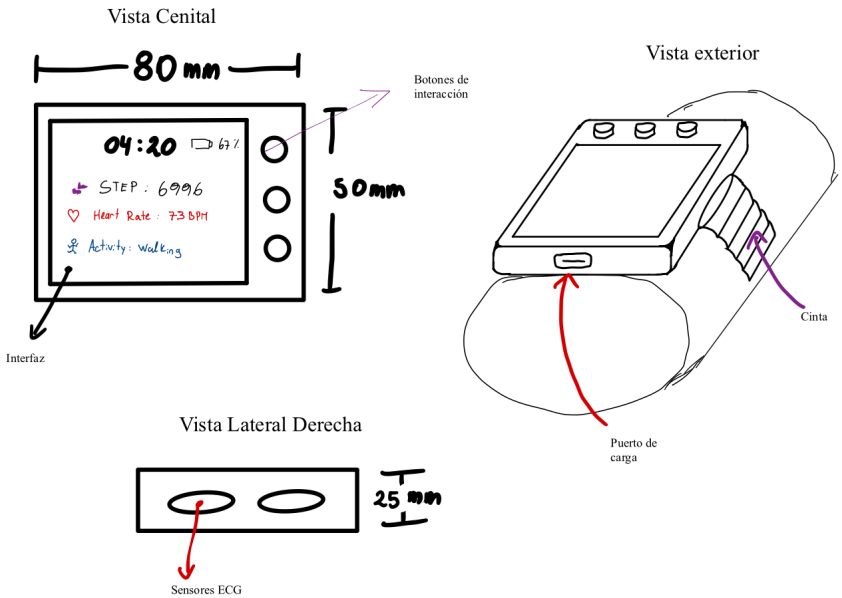

Figura 9: Bosquejo preliminar del prototipo SupaClock con vistas cenital, frontal, lateral y posterior.

8. Metodología de trabajo

Para el proyecto se colocarán los avances que cada persona va realizando en una tarea en particular, para esto se utilizará la plataforma Notion y Git (github), en las cuales se organiza la creación y ejecución de tareas, registran los avances de los proyectos de código respectivamente. También realizaremos trabajo presencial en conjunto al menos cuatro módulos a la semana se realizarán jornadas remotas orientadas principalmente para los bloques de programación y escritura de código.

8.1. Áreas y roles

El desarrollo del proyecto se divide en tres áreas principales: Hardware, Software y Procesamiento, e Interfaz y Sistema. Esta división permite un trabajo modular, paralelo y eficiente, facilitando la integración final del sistema.

8.1.1. Área 1: Hardware

La primera área es **Hardware**, esta contempla los sensores, el diseño de la PCB, el manejo físico de la energía y la integración de estos bloques. Dentro de sus tareas se encuentran la selección de componentes, diseño esquemático, diseño de PCB, integración física, diseño del sistema de alimentación y pruebas eléctricas.

A cargo de esta área se encuentra **Benjamín Sepúlveda**, encargado de registrar los avances asociados a esta rama.

Los roles asociados a esta área son:

- **Diseñador de Hardware**: encargado del diseño completo del sistema, tanto a nivel físico como eléctrico.
- **Energía**: responsable del manejo de la batería, sistema de carga y optimización del consumo energético.
- **Electrónica**: encargado de la conexión entre sensores y microcontrolador, además del debug de hardware.

8.1.2. Área 2: Software y Procesamiento

La segunda área es **Software y Procesamiento**, la cual cubre firmware, algoritmos y machine learning. Dentro de sus tareas se encuentra la lectura de sensores y su comunicación, filtrado de señales de ECG, implementación del contador de pasos y gestión del sistema operativo FreeRTOS.

A cargo de esta área se encuentra **Pablo Uribe**, encargado de la gestión de los procesos asociados.

Los roles asociados a esta área son:

- **Firmware**: encargado de la arquitectura del código, implementación de drivers y comunicación con los sensores.
- **Procesamiento y Señales**: responsable del filtrado de señales (ECG), cálculo de métricas fisiológicas y desarrollo del contador de pasos.
- **Machine Learning**: encargado de implementar y optimizar el modelo de clasificación de actividades en el dispositivo (TinyML), considerando restricciones de consumo y latencia.

8.1.3. Área 3: Interfaz y Sistema

La tercera área es **Interfaz y Sistema**, la cual contempla la interacción con el usuario, visualización de datos y comunicación externa del dispositivo. Dentro de sus tareas se encuentra el diseño de la interfaz gráfica (GUI), navegación del sistema, despliegue de métricas en pantalla, implementación de comunicación inalámbrica (BLE/WiFi) y la integración completa del sistema.

A cargo de esta área se encuentra **Tomás Avendaño**, responsable de coordinar la integración final del dispositivo.

Los roles asociados a esta área son:

- **Desarrollador UI/UX**: encargado del diseño e implementación de la interfaz gráfica, asegurando una experiencia de usuario clara e intuitiva.
- **Comunicaciones**: responsable de la transmisión de datos mediante BLE/WiFi hacia dispositivos externos o plataformas externas.
- **Integrador de Sistema**: encargado de la integración de hardware, software e interfaz, además de realizar pruebas completas del sistema.

8.2. Trabajo colaborativo

El desarrollo del proyecto se basa en una metodología de trabajo colaborativo, donde cada área cuenta con un encargado responsable de su correcta ejecución. Sin embargo, esto no implica que el trabajo recaiga exclusivamente en dicha persona, sino que su rol principal es coordinar, organizar y supervisar las tareas dentro de su área.

Cada encargado deberá distribuir los roles definidos previamente entre los integrantes del equipo, asignando responsabilidades específicas de acuerdo a las necesidades del proyecto y a las habilidades de cada miembro. De esta forma, se busca asegurar una carga de trabajo equilibrada, fomentar la especialización en distintas áreas y mejorar la eficiencia del desarrollo.

Adicionalmente, se promoverá la colaboración entre áreas, especialmente en etapas de integración, donde será necesario coordinar esfuerzos entre hardware, software e interfaz para asegurar el correcto funcionamiento del sistema completo. Este enfoque permite reducir errores de integración y facilita la detección temprana de problemas.

8.3. Asignación de tareas

La asignación de tareas se realizará de manera dinámica y progresiva, en función del avance del proyecto y los hitos establecidos en la planificación. Cada área definirá sus propias tareas específicas, las cuales serán desglosadas en actividades más pequeñas y manejables.

Para la gestión de estas tareas se utilizarán herramientas colaborativas como Notion y GitHub, las cuales permitirán llevar un seguimiento continuo del progreso, asignar responsables, establecer plazos y documentar los avances realizados. Cada tarea contará con un responsable principal, quien será el encargado de su ejecución, sin perjuicio de recibir apoyo de otros integrantes cuando sea necesario.

Asimismo, se establecerán instancias periódicas de revisión, donde cada área reportará sus avances, dificultades y próximos pasos. Esto permitirá mantener una visión global del estado del proyecto, facilitar la toma de decisiones y asegurar el cumplimiento de los objetivos planteados.

Finalmente, la asignación de tareas será flexible, permitiendo reasignaciones en caso de sobrecarga de trabajo, dificultades técnicas o cambios en los requerimientos del proyecto, garantizando así la continuidad y adaptabilidad del desarrollo.

9. Objetivos a cumplir

El desarrollo del proyecto SupaClock tiene como objetivo principal el diseño e implementación de un dispositivo wearable funcional, capaz de adquirir, procesar y visualizar variables biométricas en tiempo real, cumpliendo con los requerimientos de portabilidad, autonomía energética y robustez del sistema.

Para alcanzar este objetivo general, se plantean los siguientes objetivos específicos:

- Diseñar e implementar el sistema de adquisición de señales biométricas, incluyendo sensores de frecuencia cardíaca, SpO2, temperatura, aceleración y ECG.
- Desarrollar la arquitectura de procesamiento basada en un microcontrolador ESP32, capaz de ejecutar algoritmos de filtrado, análisis de señales y clasificación de actividad mediante Machine Learning.
- Implementar una interfaz gráfica que permita visualizar en tiempo real las métricas obtenidas, junto con un sistema de navegación intuitivo.
- Diseñar un sistema de alimentación autónomo basado en batería LiPo, incluyendo gestión de carga, regulación de voltaje y monitoreo de batería.
- Integrar todos los subsistemas en una PCB funcional, asegurando la correcta comunicación entre módulos y la estabilidad del sistema.
- Validar experimentalmente el funcionamiento del dispositivo mediante pruebas de sensores, consumo energético y desempeño del sistema completo.

9.1. Hitos

El desarrollo del proyecto se organiza en distintos hitos de avance, los cuales permiten evaluar el progreso del sistema de manera incremental.

9.1.1. Hito 5 %: Definición y planificación inicial

En esta etapa se establece la base conceptual del proyecto. Se espera contar con:

- Prueba preliminar de componentes por separado.
- Crear la base de datos en Firebase.
- Crear el servidor en Firebase.
- Crear el repositorio en GitHub.

9.1.2. Hito 25 %: Validación de subsistemas iniciales

En este punto se busca validar el funcionamiento básico de los principales módulos del sistema:

- Implementación del bloque de energía (BMS y regulación).
- Integración inicial de sensores.
- Pruebas preliminares de adquisición de datos.
- Primera integración con microcontrolador de los sensores en conjunto.
- Validación básica de algoritmos.

9.1.3. Hito 60 %: Integración funcional del sistema

En esta etapa el sistema comienza a operar de manera integrada:

- Diseño e implementación de la PCB.
- Integración de sensores en la PCB.
- Desarrollo de la interfaz gráfica en pantalla.
- Implementación de algoritmos de procesamiento (ECG, pasos).
- Integración de comunicación inalámbrica (Bluetooth).
- Pruebas funcionales del sistema completo.

9.1.4. Hito 90 %: Validación avanzada y optimización

En esta fase el sistema se encuentra prácticamente completo y se enfoca en su validación:

- Pruebas biométricas completas en condiciones reales.
- Optimización del consumo energético.
- Ajuste de algoritmos de Machine Learning.
- Validación de estabilidad del sistema.
- Integración final de hardware, software e interfaz.

9.1.5. Hito 100 %: Entrega final

Corresponde a la finalización del proyecto:

- Dispositivo completamente funcional e integrado.
- Validación final del sistema.
- Documentación completa del proyecto.
- Presentación final del prototipo.

La Fig.10 presenta la carta Gantt del proyecto, la cual desglosa las actividades planificadas para cada hito en un horizonte temporal que abarca el semestre académico completo. Las barras de color representan la duración estimada de cada tarea, permitiendo visualizar el paralelismo entre actividades de hardware, software e interfaz, así como identificar las dependencias críticas entre subsistemas.

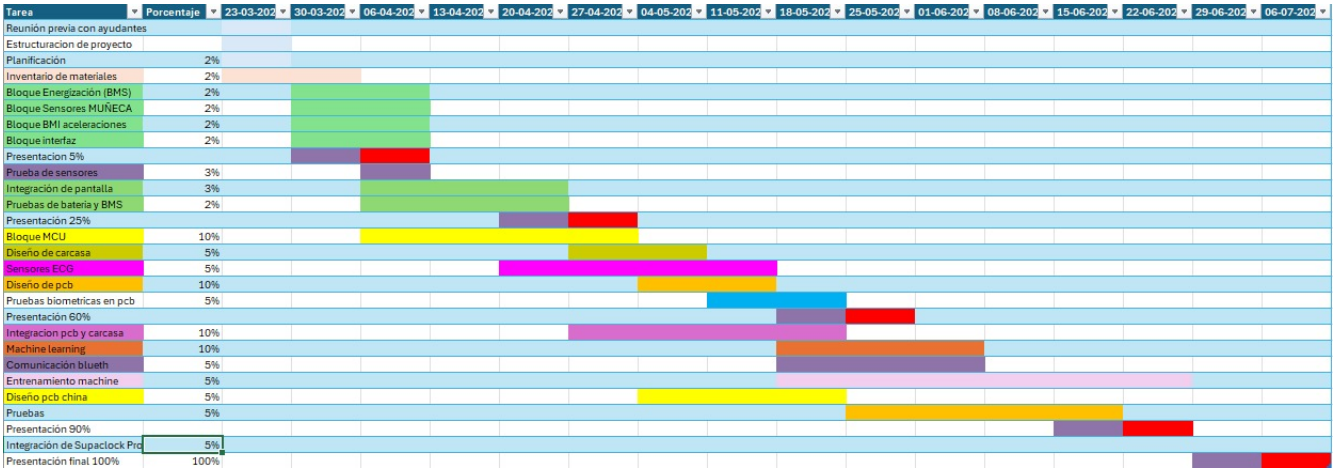

Figura 10: Carta Gantt del proyecto SupaClock con la planificación temporal de actividades por hito.

10. Presupuesto estimado

La Tabla 15 presenta el listado de materiales (*Bill of Materials*, BOM) con el costo de los componentes principales del prototipo. Los precios corresponden a las transacciones reales realizadas a través de la plataforma AliExpress durante marzo de 2026, los cuales **ya incluyen el 19 % correspondiente al IVA de importación**, junto a estimaciones conservadoras para los demás componentes genéricos. Cabe destacar que la fabricación de la placa de desarrollo prototipo es otorgada gratuitamente por el laboratorio; no obstante, para el objetivo extendido (*Bonus* "Versión Pro"), se contempla la manufactura externa (JLCPCB o PCBWay) estimando un valor de $30.000 CLP con envío incluido.

Componente	Cantidad	Referencia	Costo (CLP)
ESP32-C3 SuperMini	3	MCU principal	$8.211
ESP32-S3 SuperMini	1	MCU migración	$3.500
Módulo de expansión ESP32-C3	1	Soporte desarrollo	$2.235
MAX30102 (Módulo PPG)	2	SpO2 / HR	$3.853
MAX30205 (Sensor Temp.)	1	Temperatura clínica	$5.949
AD8232 (Módulo ECG)	1	Electrocardiograma	$3.500
BMI160 (Módulo IMU)	2	Acelerómetro + Giro	$3.204
MAX17048 (Fuel Gauge)	1	Monitor de batería	$3.000
Módulo Cargador TP4056 Type-C	1	Carga y protección	$1.142
ME6211 / AP2112K (LDO 3.3V)	1	Regulador de voltaje	$300
Pantalla IPS LCD 1.69”ST7789	1	Display principal	$3.195
Pantalla TFT LCD 1.3”	1	Display alternativo	$2.059
Conectores 24PIN FPC	10	Interfaz pantallas	$2.161
Batería LiPo 3.7V 250 mAh (502030)	1	Alimentación	$7.980
Pulsadores táctiles SMD	50	Botones navegación	$2.248
Conector USB-C hembra	1	Puerto de carga	$500
Pernos M3	10	Contactos ECG	$2.264
Componentes pasivos (R, C)	—	Varios	$1.000
Filamento PLA (carcasa 3D)	—	Impresión 3D	$2.000
PCB prototipo	1	FabLab	Gratis
Subtotal (Prototipo)	—	—	**$58.301**
PCB SMD (JLCPCB/PCBWay)	1	Versión Pro (Bonus)	$30.000
Total con Bonus	—	—	**$88.301**

Cuadro 15: Bill of Materials (BOM) con precios reales de adquisición (IVA incl.) para componentes críticos.

Informe de Avance 1 (5 %)

11. Diagramas de bloques de bajo nivel actualizados

El diagrama de bloques de alto nivel del sistema SupaClock (presentado en la Fig.4) se mantiene **sin modificaciones** respecto a la entrega de planificación. Sin embargo, tras validar la integración de los componentes y el ecosistema de desarrollo, se han diseñado diagramas de bajo nivel para detallar el funcionamiento interno de la concurrencia del firmware y el flujo de los datos biomédicos hacia la nube.

11.1. Arquitectura de Firmware (FreeRTOS)

El microcontrolador gestiona la concurrencia a través del RTOS. Para el entorno principal (`env:main`), se implementaron tareas independientes para la lectura de sensores (específicamente temperatura en esta iteración) y la actualización de la interfaz gráfica local. Las tareas están separadas lógicamente y se comunican mediante colas de mensajes estructuradas (`xQueue`), garantizando la concurrencia segura. La tarea encargada de las notificaciones BLE (`TaskBLE`) se encuentra delineada en la arquitectura, pero no se ha activado ya que la aplicación móvil no es parte del alcance técnico de esta entrega.

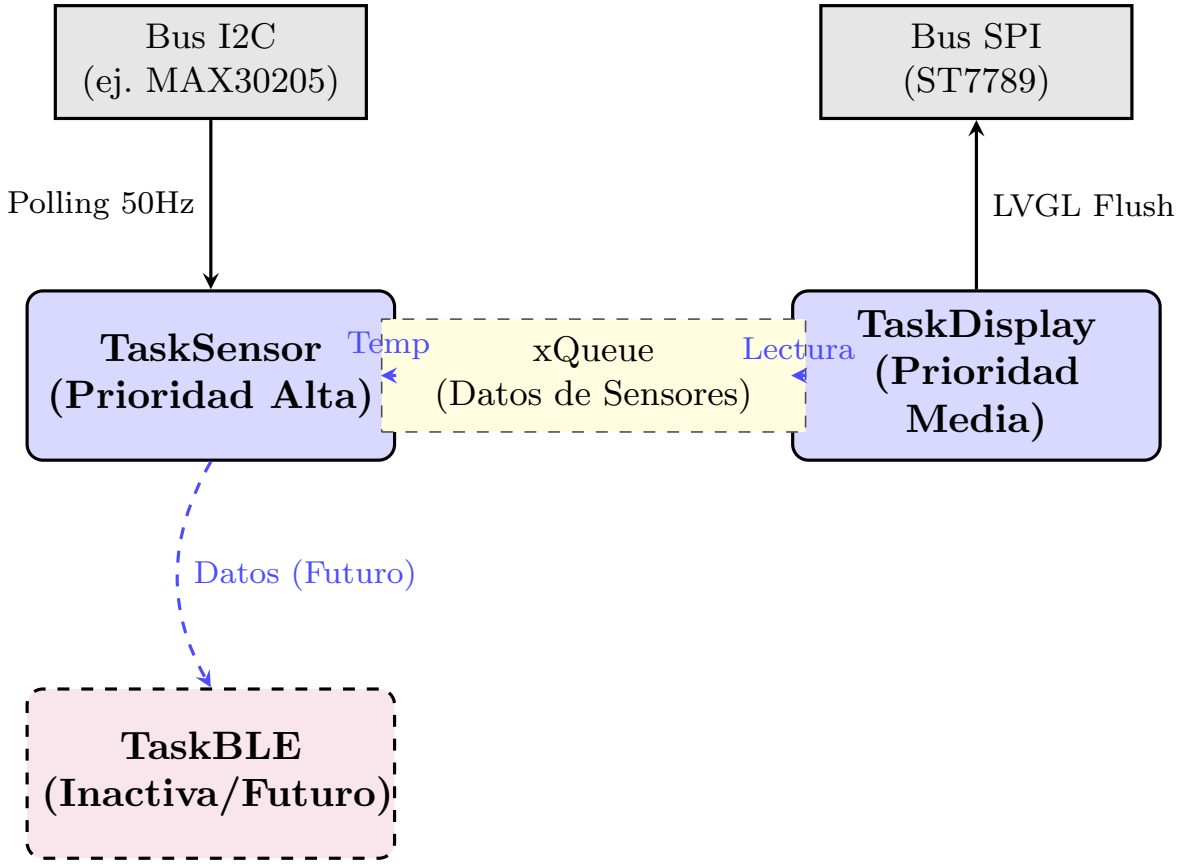

Figura 11: Arquitectura de tareas concurrentes (FreeRTOS) implementada en el ESP32-C3, destacando el flujo de lectura de sensores y visualización gráfica local.

11.2. Pipeline de Datos Biomédicos

El flujo de datos desde la captura en hardware hasta el procesamiento analítico en la nube se modela en una tubería (*pipeline*). A pesar de que los datos biomédicos complejos del ECG y la fotopletismografía aún no entran en funcionamiento, la arquitectura subyacente para su ingesta, digitalización, transmisión BLE y almacenamiento en Firebase está completamente definida en el backend implementado.

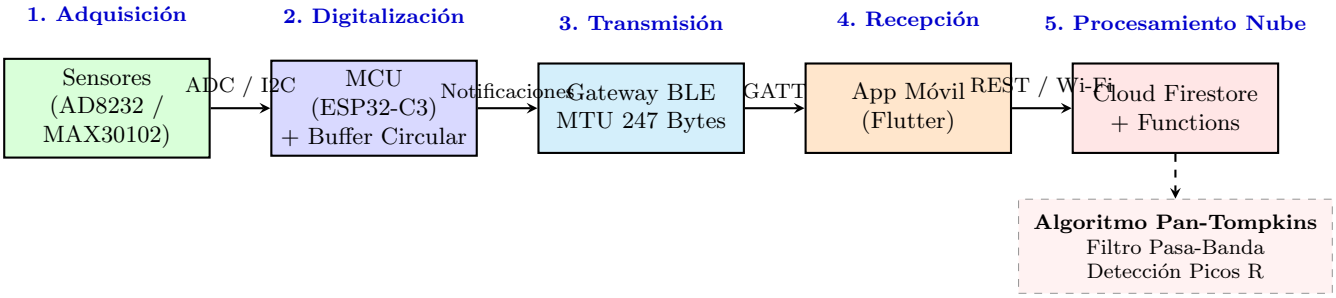

Figura 12: Pipeline de adquisición y procesamiento de datos biomédicos, modelando hacia el futuro el recorrido completo de la señal.

12. Planificación actualizada

12.1. Estado de avance respecto a la planificación

La Tabla 16 presenta el contraste entre las tareas planificadas para el hito del 5 % y el estado actual de ejecución.

Tarea planificada (5 %)	Estado	Observaciones
Prueba preliminar de componentes	**Completado**	Se probaron: ST7789, MAX30205, MAX30102 y BMI160 de forma individual.
Crear base de datos en Firebase	**Completado**	Cloud Firestore desplegado con estructura de colecciones y reglas de seguridad.
Crear servidor en Firebase	**Completado**	Cloud Functions desplegadas con el algoritmo Pan-Tompkins [16] en Python.
Crear repositorio en GitHub	**Completado**	Repositorio activo con estructura modular PlatformIO y múltiples entornos de test.
Driver de pantalla LCD (adelantado)	**Completado**	Driver ST7789 funcional con SPI a 80 MHz y soporte DMA. Integración con LVGL 8.4 [36].
Decodificación JPEG en MCU (adelantado)	**Completado**	Algoritmo JPEGDEC [37] integrado con SPIFFS [38] para renderizar imágenes directamente en pantalla.
Prueba de BLE con NimBLE (adelantado)	**Completado**	Stack BLE funcional con servicios GATT, bonding y notificaciones de telemetría simulada [39].

Cuadro 16: Contraste entre avance esperado (5 %) y avance logrado. Las filas amarillas indican tareas adelantadas respecto a la planificación original.

Como se observa, todas las tareas planificadas para el hito del 5 % han sido completadas satisfactoriamente. Adicionalmente, se realizaron avances no previstos originalmente para este hito (resaltados en amarillo), como la integración completa del driver de pantalla con LVGL, la decodificación de imágenes JPEG en el microcontrolador y la implementación del stack BLE con NimBLE. Estos avances adelantan trabajo correspondiente al hito del 25 %.

12.2. Actualización de la Carta Gantt

La carta Gantt presentada en la Fig.10 se mantiene como referencia base. Respecto a ella, se registran los siguientes ajustes producto del avance real:

- **Tareas adelantadas:** Las actividades de "Driver de pantalla", "Integración LVGL" y "Prueba BLE" originalmente planificadas para el hito del 25 % fueron completadas durante el hito del 5 %, ya que resultaron ser prerrequisitos naturales para validar la arquitectura de firmware.
- **Tareas repriorizadas:** La "Integración inicial de sensores en conjunto" (planificada para 25 %) se mantiene en su fecha original, ya que requiere la implementación previa del subsistema de energía (BMS + LDO) para operar los sensores de forma autónoma sin alimentación USB.
- **Sin atrasos:** No se reportan tareas atrasadas respecto a la planificación original.

12.3. Distribución del trabajo

Para esta primera etapa, el equipo adoptó un enfoque de **trabajo colaborativo integral**, donde los tres integrantes participaron activamente en todas las áreas del proyecto. La distribución de roles fue la siguiente:

- **Tomás Avendaño:** Participó en las pruebas de sensores (MAX30205, MAX30102), implementación del stack BLE con NimBLE y despliegue del backend Firebase.
- **Benjamín Sepúlveda:** Contribuyó en el cableado de prototipos, verificación de niveles de voltaje en protoboard, pruebas del bus I2C y documentación del informe.
- **Pablo Uribe:** Participó en la implementación del driver ST7789, integración de JPEGDEC y LVGL, y actuó como **verificador del trabajo conjunto**, asegurando la coherencia entre los módulos desarrollados por cada integrante.

13. Implementación y resultados

En esta sección se describen los bloques implementados durante este primer hito de avance. Cada subsección detalla la funcionalidad del bloque, las pruebas realizadas y los resultados obtenidos.

13.1. Bloque de Interfaz Local: Driver de pantalla ST7789

13.1.1. Descripción y funcionalidad

Se implementó un driver completo en C para el controlador de pantalla ST7789 [11] (pantalla IPS LCD de 1.69", 240×280 píxeles, RGB565). El driver opera sobre el bus SPI del ESP32-C3 [4] a una velocidad de reloj de **80 MHz**, utilizando DMA automático (`SPI_DMA_CH_AUTO`) para transferencias asíncronas que liberan al procesador durante el volcado de píxeles.

Las funciones principales implementadas son:

- `st7789_init()`: Configura el bus SPI, ejecuta la secuencia de inicialización del controlador (software reset, sleep out, inversión de color para IPS, modo RGB565 de 16 bits) y enciende la pantalla.
- `st7789_fill_screen(color)`: Pinta la pantalla completa de un color uniforme, enviando los datos en bloques de 40 líneas.
- `st7789_draw_bitmap(x, y, w, h, bitmap)`: Renderiza un bloque rectangular de píxeles en la posición especificada, con recorte automático en los bordes para evitar artefactos (*ghosting*).

Un detalle técnico relevante es la compensación de *offset* vertical de 20 píxeles en la función `set_addr_window()`, necesaria porque la RAM interna del ST7789 tiene una resolución de 320 líneas [11], pero la pantalla física solo expone 280.

13.1.2. Integración con LVGL 8.4

Se integró la librería LVGL versión 8.4 [36]. Se implementó un callback `flush_cb` que conecta el lienzo virtual de LVGL con `st7789_draw_bitmap()`, permitiendo renderizar widgets (etiquetas, barras de progreso, animaciones) de forma eficiente.

El buffer de dibujo parcial se configuró en **14.4 KB** (240 × 30 líneas × 2 bytes/píxel), permitiendo un uso eficiente de la SRAM del ESP32-C3 (400 KB) [4].

13.1.3. Resultados

Se verificó el correcto funcionamiento mostrando textos con anti-aliasing, barras de progreso animadas y colores vibrantes a ∼30 FPS.

13.2. Bloque de Interfaz Local: Decodificación JPEG (JPEGDEC)

13.2.1. Descripción y funcionalidad

Se integró la librería **JPEGDEC** [37] para decodificar imágenes JPEG almacenadas en la partición SPIFFS [38] del ESP32 directamente hacia la pantalla, sin almacenar el bitmap descomprimido en RAM. JPEGDEC decodifica en bloques MCU de 16×16 píxeles, invocando un callback (`JPEGDraw`) que envía cada bloque al ST7789 vía `st7789_draw_bitmap()`.

Este enfoque de *streaming* es fundamental: una imagen de 240×280 en RGB565 ocuparía 134.4 KB descomprimida (más del 30 % de la SRAM), mientras que el JPEG comprimido ocupa 10–20 KB en flash con un overhead de RAM de solo ∼4 KB.

13.2.2. Resultados

Se logró decodificar y mostrar imágenes JPEG de 240×280 almacenadas en SPIFFS. El tiempo de renderizado completo es de 200–400 ms.

13.3. Bloque de Sensores: Prueba del MAX30102 (PPG)

13.3.1. Descripción y funcionalidad

Se implementó un driver inicial para el sensor fotopletismógrafo **MAX30102** [7], operando en el bus I2C a la dirección `0x57`. El driver implementa inicialización (software reset, modo SpO2, ajuste de LEDs) y lectura de la FIFO (muestras de 18 bits para canales rojo e IR).

La lectura de datos accede al registro `FIFO_DATA (0x07)` [7], donde cada muestra consiste en 6 bytes (3 por canal), enmascarados a 18 bits significativos.

13.3.2. Resultados

Se verificó la comunicación I2C confirmando la lectura correcta del registro `PART_ID (0xFF) = 0x15` [7]. Se obtuvieron lecturas crudas de señales roja e IR con variación al colocar el dedo sobre el sensor. La calibración algorítmica para SpO2 y BPM será implementada en la siguiente etapa.

13.4. Bloque de Comunicación: BLE con NimBLE

13.4.1. Descripción y funcionalidad

Se implementó el stack Bluetooth Low Energy utilizando la librería **Apache NimBLE** [39] integrada en ESP-IDF. Se configuraron dos servicios GATT:

- **Device Information Service (0x180A):** Expone el nombre del fabricante ("ArticunoFruna") y el modelo ("SupaClock-C3") como características de solo lectura.

- **SupaClock Telemetry Service (UUID personalizado):** Contiene tres características con notificaciones: HR/SpO2, Temperatura y Pasos.

Se configuró el *Appearance* como "Wristwatch" (0x00C1) según la especificación BLE [15], y se habilitó *Secure Connections* (SC) con bonding para persistir el emparejamiento entre sesiones. La publicidad BLE se dividió en dos paquetes: el principal con flags y UUID del servicio, y el *Scan Response* con el nombre del dispositivo.

13.4.2. Resultados

El dispositivo es visible y emparejable desde cualquier teléfono Android/iOS. Se verificó la recepción de notificaciones con datos de telemetría simulada (dummies) a 1 Hz.

13.5. Bloque de Backend: Firebase y Cloud Firestore

13.5.1. Descripción y funcionalidad

Se desplegó el ecosistema de backend utilizando **Firebase** [14]:

Cloud Firestore se configuró con la siguiente jerarquía:

- `users/{userId}`: Perfil del usuario.
- `users/.../sessions/{sessionId}`: Sesiones con métricas agregadas.
- `users/.../ecgReadings/{readingId}`: Lecturas de ECG crudas y procesadas.

Cloud Functions (Python 3.12) implementa:

- `process_ecg`: Trigger automático que ejecuta el pipeline **Pan-Tompkins** [16] completo (filtrado pasabanda, derivación, cuadratura, integración de ventana móvil, detección de picos R con umbral adaptativo). Calcula BPM y HRV (SDNN).
- `compute_daily_summary`: Función callable que agrega métricas diarias.

13.5.2. Resultados

Se verificó el correcto despliegue mediante pruebas con datos sintéticos. El algoritmo Pan-Tompkins detecta correctamente picos R y calcula BPM dentro de rangos fisiológicamente plausibles (60–100 BPM para adulto en reposo simulado).

13.6. Evaluación general y próximos pasos

Los resultados de este primer hito validan la viabilidad de la arquitectura. Todos los componentes probados individualmente operan correctamente a través de las interfaces planificadas (I2C y SPI). El backend está operativo y listo para recibir datos reales.

Para el siguiente hito (25 %), los esfuerzos se concentrarán en:

- Integración simultánea de múltiples sensores en el bus I2C (con mutex FreeRTOS [6]).
- Implementación del subsistema de energía (BMS TP4056 [25] + LDO + Fuel Gauge).
- Desarrollo del algoritmo de conteo de pasos sobre datos reales del BMI160 [10].

- Inicio de la recolección de datos de entrenamiento para el modelo de ML.

14. Referencias

Referencias

[1] Jude Nwadiuto Njoku, Manuel Estrada, Sebastian Ma, Zixian Zhong, Rafi Ibn Sultan Chowdhury, et al. Deep learning-based human activity recognition on edge devices: A benchmark analysis. In *IEEE International Conference on Communications (ICC)*, 2024. Disponible en: https://arxiv.org/abs/2307.09424.

[2] CIPER Chile. Resultados de la encuesta nacional de actividad física y deporte: una oportunidad para fortalecer el bienestar integral en chile, 2025. Disponible en: https://www.ciperchile.cl/2025/05/24/resultados-de-la-encuesta-nacional-de-actividad-fisica-y-deporte/.

[3] UC Christus. 5 razones por las que chile es el país con más obesidad de latinoamérica, 2025. Disponible en: https://www.ucchristus.cl/blog-salud-uc/articulos/2025/5-razones-por-las-que-chile-es-el-pais-con-mas-obesidad-de-latinoamerica.

[4] Espressif Systems. ESP32-C3 Series Datasheet, 2024. Disponible en: https://www.espressif.com/sites/default/files/documentation/esp32-c3_datasheet_en.pdf.

[5] Espressif Systems. ESP32-S3 Series Datasheet, 2024. Disponible en: https://www.espressif.com/sites/default/files/documentation/esp32-s3_datasheet_en.pdf.

[6] Amazon Web Services. *FreeRTOS Reference Manual*, 2024. Disponible en: https://www.freertos.org/Documentation/RTOS_book.html.

[7] Analog Devices (Maxim Integrated). MAX30102: High-Sensitivity Pulse Oximeter and Heart-Rate Sensor for Wearable Health, 2018. Disponible en: https://www.analog.com/media/en/technical-documentation/data-sheets/MAX30102.pdf.

[8] Analog Devices (Maxim Integrated). MAX30205: Human Body Temperature Sensor, 2016. Disponible en: https://www.analog.com/media/en/technical-documentation/data-sheets/MAX30205.pdf.

[9] Analog Devices. AD8232: Single-Lead, Heart Rate Monitor Front End, 2012. Disponible en: https://www.analog.com/media/en/technical-documentation/data-sheets/ad8232.pdf.

[10] Bosch Sensortec. BMI160: Small, Low Power Inertial Measurement Unit, 2020. Disponible en: https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bmi160-ds000.pdf.

[11] Sitronix Technology. ST7789V: 240RGB x 320 Dot 262K Color with Frame Memory Single-Chip TFT Controller/Driver, 2014. Disponible en: https://www.newhavendisplay.com/appnotes/datasheets/LCDs/ST7789V.pdf.

[12] Edge Impulse Inc. *Edge Impulse Documentation*, 2024. Disponible en: https://docs.edgeimpulse.com/.

[13] Google. *TensorFlow Lite for Microcontrollers*, 2024. Disponible en: https://www.tensorflow.org/lite/microcontrollers.

[14] Google. *Firebase Documentation: Cloud Firestore and Cloud Functions*, 2024. Disponible en: https://firebase.google.com/docs.

[15] Bluetooth SIG. Bluetooth Core Specification v5.3, 2021. Disponible en: https://www.bluetooth.com/specifications/specs/core-specification-5-3/.

[16] Jiapu Pan and Willis J. Tompkins. A real-time qrs detection algorithm. *IEEE Transactions on Biomedical Engineering*, BME-32(3):230–236, 1985. doi: 10.1109/TBME.1985.325532.

[17] Nordic Semiconductor. nRF52840 Product Specification, 2021. Disponible en: https://docs.nordicsemi.com/bundle/nRF52840_PS_v1.8/page/keyfeatures_html5.html.

[18] STMicroelectronics. STM32WB55xx Datasheet, 2022. Disponible en: https://www.st.com/resource/en/datasheet/stm32wb55cg.pdf.

[19] InvenSense (TDK). MPU-6050: Six-Axis MEMS MotionTracking Device, 2013. Disponible en: https://invensense.tdk.com/wp-content/uploads/2015/02/MPU-6000-Datasheet1.pdf.

[20] STMicroelectronics. LSM6DS3: iNEMO inertial module, 2017. Disponible en: https://www.st.com/resource/en/datasheet/lsm6ds3.pdf.

[21] Analog Devices (Maxim Integrated). MAX30100: Pulse Oximeter and Heart-Rate Sensor IC for Wearable Health, 2014. Disponible en: https://www.analog.com/media/en/technical-documentation/data-sheets/MAX30100.pdf.

[22] Analog Devices (Maxim Integrated). MAX86141: Optical Data Acquisition System, 2019. Disponible en: https://www.analog.com/media/en/technical-documentation/data-sheets/MAX86140-MAX86141.pdf.

[23] Melexis. MLX90614 Family: Single and Dual Zone Infra Red Thermometer in TO-39, 2019. Disponible en: https://www.melexis.com/-/media/files/documents/datasheets/mlx90614-datasheet-melexis.pdf.

[24] Analog Devices (Maxim Integrated). DS18B20: Programmable Resolution 1-Wire Digital Thermometer, 2019. Disponible en: https://www.analog.com/media/en/technical-documentation/data-sheets/DS18B20.pdf.

[25] NanJing Top Power ASIC Corp. TP4056: 1A Standalone Linear Li-Ion Battery Charger with Thermal Regulation, 2008. Disponible en: https://dlnmh9ip6v2uc.cloudfront.net/datasheets/Prototyping/TP4056.pdf.

[26] Texas Instruments. bq2407x Standalone 1-Cell 1.5-A Linear Battery Charger with Power Path Management, 2019. Disponible en: https://www.ti.com/lit/ds/symlink/bq24075.pdf.

[27] Microchip Technology. MCP73831/2: Miniature Single-Cell, Fully Integrated Li-Ion, Li-Polymer Charge Management Controllers, 2014. Disponible en: https://ww1.microchip.com/downloads/en/DeviceDoc/20001984g.pdf.

[28] Analog Devices (Maxim Integrated). MAX17048/MAX17049: Micropower 1-Cell/2-Cell Li+ Fuel Gauge with ModelGauge, 2013. Disponible en: https://www.analog.com/media/en/technical-documentation/data-sheets/MAX17048-MAX17049.pdf.

[29] Micro One Electronic Inc. ME6211: High Speed LDO Regulators, 2019. Disponible en: https://datasheet.lcsc.com/lcsc/1811131510_MICRONE-Nanjing-Micro-One-Elec-ME6211C33M5G-N_C82942.pdf.

[30] Analog Devices (Maxim Integrated). MAX17043/MAX17044: Compact, Low-Cost 1S/2S Fuel Gauges with ModelGauge, 2014. Disponible en: https://www.analog.com/media/en/technical-documentation/data-sheets/MAX17043-MAX17044.pdf.

[31] Texas Instruments. bq27441-G1 System-Side Impedance Track Fuel Gauge, 2018. Disponible en: https://www.ti.com/lit/ds/slusbq8b/slusbq8b.pdf.

[32] Advanced Monolithic Systems. AMS1117: 1A LOW DROPOUT VOLTAGE REGULATOR, 2006. Disponible en: http://www.advanced-monolithic.com/pdf/ds1117.pdf.

[33] Texas Instruments. TPS7A02 200-mA, Nanopower IQ (25 nA), Low-Dropout Regulator, 2022. Disponible en: https://www.ti.com/lit/ds/symlink/tps7a02.pdf.

[34] Solomon Systech. SSD1306: 128 x 64 Dot Matrix OLED/PLED Segment/Common Driver with Controller, 2008. Disponible en: https://cdn-shop.adafruit.com/datasheets/SSD1306.pdf.

[35] Sharp Microelectronics. LS013B7DH03: Memory LCD Data Sheet, 2014. Disponible en: https://www.sharpsma.com/documents/20181/0/LS013B7DH03_Spec.pdf.

[36] LVGL LLC. *LVGL: Light and Versatile Graphics Library, v8.4*, 2024. Disponible en: https://docs.lvgl.io/8.4/.

[37] Larry Bank. JPEGDEC: An optimized JPEG decoder for Arduino and other embedded platforms, 2023. Disponible en: https://github.com/bitbank2/JPEGDEC.

[38] Espressif Systems. *ESP-IDF SPIFFS Filesystem Documentation*, 2024. Disponible en: https://docs.espressif.com/projects/esp-idf/en/stable/esp32c3/api-reference/storage/spiffs.html.

[39] Apache Software Foundation. *Apache Mynewt NimBLE: Bluetooth Low Energy Stack*, 2024. Disponible en: https://mynewt.apache.org/latest/network/.

15. Anexos

15.1. Anexo A: Código del driver ST7789 (extracto principal)

```
esp_err_t st7789_init(void) {
    // GPIO Config para pines DC y RST
    gpio_config_t gc = {
        .pin_bit_mask = (1ULL << ST7789_DC_PIN)
                      | (1ULL << ST7789_RST_PIN),
        .mode = GPIO_MODE_OUTPUT,
    };
    gpio_config(&gc);

    // SPI Bus a 80 MHz con DMA automatico
    spi_bus_config_t buscfg = {
        .mosi_io_num = ST7789_MOSI_PIN,
        .sclk_io_num = ST7789_SCK_PIN,
        .max_transfer_sz = 240 * 280 * 2 + 8
    };
    spi_bus_initialize(SPI2_HOST, &buscfg,
                       SPI_DMA_CH_AUTO);

    // Hardware Reset + Init Sequence
    lcd_cmd(spi, ST7789_SWRESET);
    lcd_cmd(spi, ST7789_SLPOUT);
    lcd_cmd(spi, ST7789_INVON); // IPS requiere inversion
    lcd_cmd(spi, ST7789_DISPON);
    return ESP_OK;
}
```

Listing 1: Función de inicialización del driver ST7789.

15.2. Anexo B: Callback de decodificación JPEG

```
// Callback que envia bloques 16x16 decodificados
// directamente al ST7789 sin almacenar en RAM
int JPEGDraw(JPEGDRAW *pDraw) {
    st7789_draw_bitmap(
        pDraw->x, pDraw->y,
        pDraw->iWidth, pDraw->iHeight,
        pDraw->pPixels
    );
    return 1; // Continuar decodificacion
}

// Uso: decodificar desde SPIFFS
jpeg.open("/spiffs/foto.jpg", myOpen, myClose,
          myRead, mySeek, JPEGDraw);
jpeg.setPixelType(RGB565_BIG_ENDIAN);
jpeg.decode(0, 0, 0);
```

Listing 2: Callback de renderizado JPEG por bloques MCU.

15.3. Anexo C: Driver MAX30102 (lectura FIFO)

```
esp_err_t max30102_read_fifo(uint32_t *red,
                             uint32_t *ir) {
    uint8_t reg = MAX30102_REG_FIFO_DATA;
    uint8_t data[6];
    esp_err_t err = i2c_master_write_read_device(
        I2C_MASTER_NUM, MAX30102_I2C_ADDR,
        &reg, 1, data, 6,
        I2C_MASTER_TIMEOUT_MS / portTICK_PERIOD_MS
    );
    if (err == ESP_OK) {
        *red = ((uint32_t)data[0] << 16)
             | ((uint32_t)data[1] << 8) | data[2];
        *red &= 0x03FFFF; // 18-bit data
        *ir  = ((uint32_t)data[3] << 16)
             | ((uint32_t)data[4] << 8) | data[5];
        *ir  &= 0x03FFFF;
    }
    return err;
}
```

Listing 3: Lectura de muestras desde la FIFO del MAX30102.

15.4. Anexo D: Extracto de Cloud Functions (Pan-Tompkins)

```
@firestore_fn.on_document_created(
    document="users/{userId}/sessions/"
             "{sessionId}/ecgReadings/{readingId}",
    region="us-central1",
    memory=options.MemoryOption.MB_256,
)
def process_ecg(event):
    data = event.data.to_dict()
```

```
    raw_data = data.get("rawData", [])
    fs = data.get("samplingRate", 100.0)

    # Pipeline Pan-Tompkins completo
    results = pan_tompkins(raw_data, fs)

    # Actualizar documento con resultados
    event.data.reference.update({
        "processedBPM": results["bpm"],
        "hrv": results["hrv"],
        "rPeaks": results["r_peaks"],
        "processingStatus": "completed",
    })
```

Listing 4: Trigger de procesamiento de ECG en Cloud Functions.